baseline/model.py — Flight Reading Key Sections

配套文档: docs/10-research/flight-reading-list-2026-05-06.md + docs/00-context/model-py-section-map.md
重点段落； 核心必读； EXP-013 关键差异源头

Table of Contents

01. §1 ModelInput (数据结构) (L11-L19) ← —
02. §2 RotaryEmbedding + rotate_half + apply_rope_to_tensor (L26-L92) ← 论文 2 RoFormer §3.4
03. §3 SwiGLU (L100-L114) ← LLaMA/PaLM 系列惯例，baseline 默认激活
04. §4 RoPEMultiheadAttention（L117-L243) ← 论文 1 Attention + 论文 2 RoFormer
05. §5 CrossAttention (L244-L314) ← HyFormer Query Decoding
06. §6 RankMixerBlock（Token Mixing 灵魂) (L315-L414) ← 论文 4 RankMixer
07. §7 MultiSeqQueryGenerator (L415-L557) ← 论文 3 HyFormer §3.4 Multi-Sequence
08. §8 SwiGLUEncoder (baseline 默认，3.5× 更快) (L558-L598) ← —
09. §9 TransformerEncoder (对照) (L600-L671) ← 论文 1 Attention
10. §10 LongerEncoder (L672-L905) ← 论文 7 LONGER
11. §11 MultiSeqHyFormerBlock（HyFormer 核心 block) (L906-L1043) ← 论文 3 HyFormer §3.2-3.4
12. §12 OneTransMixedAttn（Mixed Parameterization) (L1044-L1217) ← 论文 5 OneTrans §Mixed Attn
13. §13 OneTransMixedFFN (L1218-L1280) ← 论文 5 OneTrans §Mixed FFN
14. §14 OneTransNSConditioner（EXP-013 +0.010 AUC 源头) (L1281-L1349) ← 仓库自研扩展（有创新奖潜质）
15. §15 OneTransBlock (L1350-L1484) ← 论文 5 OneTrans (整体装配)
16. §16 GroupNSTokenizer (原 baseline 方式) (L1485-L1566) ← —
17. §17 RankMixerNSTokenizer (当前默认) (L1567-L1688) ← 论文 4 RankMixer NS 部分
18. §18 PCVRHyFormer（顶层装配) (L1689-L2396) ← 整体装配（所有论文的综合体现）

01. §1 ModelInput (数据结构)

Lines 11-19 (9 lines) · Paper ref: —

飞机上读的时候重点关注：

- 理解模型输入每个字段的 shape

```
11 | class ModelInput(NamedTuple):
12 |     user_int_feats: torch.Tensor
13 |     item_int_feats: torch.Tensor
14 |     user_dense_feats: torch.Tensor
15 |     item_dense_feats: torch.Tensor
16 |     seq_data: dict        # {domain: tensor [B, S, L]}
17 |     seq_lens: dict        # {domain: tensor [B]}
18 |     seq_time_buckets: dict  # {domain: tensor [B, L]}
19 |
```

02. §2 RotaryEmbedding + rotate_half + apply_rope_to_tensor

Lines 26-92 (67 lines) · Paper ref: 论文 2 RoFormer §3.4

飞机上读的时候重点关注：

- inv_freq = 1/base^(2i/d) 即论文的 θ_i
- _build_cache 预计算 cos/sin
- rotate_half 是论文公式 (15) 的 pair-interleave 简化

```
26 | class RotaryEmbedding(nn.Module):
27 |     """Precomputes and caches RoPE cos/sin values.
28 |
29 |     Attributes:
30 |         dim: Rotary embedding dimension.
31 |         max_seq_len: Maximum sequence length for cache.
32 |         base: Base frequency for rotary encoding.
33 |     """
34 |
35 |     def __init__(self, dim: int, max_seq_len: int = 2048, base: float = 10000.0) -> None:
36 |         super().__init__()
37 |         self.dim = dim
38 |         self.max_seq_len = max_seq_len
39 |         self.base = base
40 |
41 |         # Precompute inv_freq: (dim // 2,)
42 |         inv_freq = 1.0 / (base ** (torch.arange(0, dim, 2).float() / dim))
43 |         self.register_buffer('inv_freq', inv_freq, persistent=False)
44 |
45 |         # Precompute cache
46 |         self._build_cache(max_seq_len)
47 |
48 |     def _build_cache(self, seq_len: int) -> None:
49 |         t = torch.arange(seq_len, dtype=self.inv_freq.dtype, device=self.inv_freq.device)
50 |         freqs = torch.outer(t, self.inv_freq)  # (seq_len, dim // 2)
51 |         emb = torch.cat([freqs, freqs], dim=-1)  # (seq_len, dim)
52 |         self.register_buffer('cos_cached', emb.cos().unsqueeze(0), persistent=False)  # (1, seq_len, dim)
53 |         self.register_buffer('sin_cached', emb.sin().unsqueeze(0), persistent=False)  # (1, seq_len, dim)
54 |
55 |     def forward(self, seq_len: int, device: torch.device) -> Tuple[torch.Tensor, torch.Tensor]:
56 |         """Computes cos/sin values for the given sequence length.
57 |
58 |         Returns pre-computed slices from the cache. The cache is built once
59 |         in __init__ with max_seq_len; no runtime expansion is performed so
60 |         that the forward pass remains compatible with torch.compile().
61 |         """
62 |         cos = self.cos_cached[:, :seq_len, :].to(device)
63 |         sin = self.sin_cached[:, :seq_len, :].to(device)
64 |         return cos, sin
65 |
66 |
67 | def rotate_half(x: torch.Tensor) -> torch.Tensor:
68 |     """Swaps and negates the first and second halves of the last dimension."""
69 |     x1 = x[..., : x.shape[-1] // 2]
70 |     x2 = x[..., x.shape[-1] // 2 :]
71 |     return torch.cat([-x2, x1], dim=-1)
72 |
73 |
74 | def apply_rope_to_tensor(
75 |     x: torch.Tensor,
76 |     cos: torch.Tensor,
77 |     sin: torch.Tensor,
78 | ) -> torch.Tensor:
79 |     """Applies Rotary Position Embedding to a single tensor.
80 |
81 |     Args:
82 |         x: (B, num_heads, L, head_dim)
83 |         cos: (1, L_max, head_dim) or (B, L, head_dim) for batch-specific positions.
84 |         sin: Same shape as cos.
85 |
86 |     Returns:
87 |         Rotated tensor of shape (B, num_heads, L, head_dim).
88 |     """
89 |     L = x.shape[2]
90 |     cos_ = cos[:, :L, :].unsqueeze(1)  # (*, 1, L, head_dim)
91 |     sin_ = sin[:, :L, :].unsqueeze(1)
92 |     return x * cos_ + rotate_half(x) * sin_
```

03. §3 SwiGLU

Lines 100-114 (15 lines) · Paper ref: LLaMA/PaLM 系列惯例，baseline 默认激活

飞机上读的时候重点关注：

- 为什么 SiLU 比 ReLU/GELU 更稳定
- gate 后为什么再一次 Linear（fc_out）

```
100 | class SwiGLU(nn.Module):
101 |     """SwiGLU activation: x1 * SiLU(x2)."""
102 |
103 |     def __init__(self, d_model: int, hidden_mult: int = 4) -> None:
104 |         super().__init__()
105 |         hidden_dim = d_model * hidden_mult
106 |         self.fc = nn.Linear(d_model, 2 * hidden_dim)
107 |         self.fc_out = nn.Linear(hidden_dim, d_model)
108 |
109 |     def forward(self, x: torch.Tensor) -> torch.Tensor:
110 |         x = self.fc(x)
111 |         x1, x2 = x.chunk(2, dim=-1)
112 |         x = x1 * F.silu(x2)
113 |         x = self.fc_out(x)
114 |         return x
```

04. §4 RoPEMultiheadAttention

Lines 117-243 (127 lines) · Paper ref: 论文 1 Attention + 论文 2 RoFormer

飞机上读的时候重点关注：

- W_q/W_k/W_v/W_o 四投影 + W_g 是仓库扩展（gate）
- forward 里 reshape (B, L, D) → (B, h, L, d/h) 的维度变换
- RoPE 是在 Q/K 投影完成 reshape 后、dot-product 之前注入
- F.scaled_dot_product_attention 是 PyTorch 2.0+ 融合核

```
117 | class RoPEMultiheadAttention(nn.Module):
118 |     """Multi-head attention with Rotary Position Embedding support.
119 |
120 |     Manually projects Q/K/V and reshapes for multi-head, then injects RoPE
121 |     after projection and before dot-product. Uses F.scaled_dot_product_attention
122 |     for efficient computation.
123 |     """
124 |
125 |     def __init__(
126 |         self,
127 |         d_model: int,
128 |         num_heads: int,
129 |         dropout: float = 0.0,
130 |         rope_on_q: bool = True,
131 |     ) -> None:
132 |         super().__init__()
133 |         self.d_model = d_model
134 |         self.num_heads = num_heads
135 |         self.head_dim = d_model // num_heads
136 |         self.rope_on_q = rope_on_q
137 |         self.dropout = dropout
138 |
139 |         assert d_model % num_heads == 0, "d_model must be divisible by num_heads"
140 |
141 |         self.W_q = nn.Linear(d_model, d_model)
142 |         self.W_k = nn.Linear(d_model, d_model)
143 |         self.W_v = nn.Linear(d_model, d_model)
144 |         self.W_o = nn.Linear(d_model, d_model)
145 |         self.W_g = nn.Linear(d_model, d_model)
146 |
147 |         nn.init.zeros_(self.W_g.weight)
148 |         nn.init.constant_(self.W_g.bias, 1.0)
149 |
150 |     def forward(
151 |         self,
152 |         query: torch.Tensor,
153 |         key: torch.Tensor,
154 |         value: torch.Tensor,
155 |         key_padding_mask: Optional[torch.Tensor] = None,
156 |         attn_mask: Optional[torch.Tensor] = None,
157 |         rope_cos: Optional[torch.Tensor] = None,
158 |         rope_sin: Optional[torch.Tensor] = None,
159 |         q_rope_cos: Optional[torch.Tensor] = None,
160 |         q_rope_sin: Optional[torch.Tensor] = None,
161 |         need_weights: bool = False,
162 |     ) -> tuple:
163 |         """Computes multi-head attention with optional RoPE.
164 |
165 |         Args:
166 |             query: (B, Lq, D)
167 |             key: (B, Lk, D)
168 |             value: (B, Lk, D)
169 |             key_padding_mask: (B, Lk), True indicates padding positions.
170 |             attn_mask: (Lq, Lk) or (B*num_heads, Lq, Lk), additive mask.
171 |             rope_cos: (1, L, head_dim), RoPE for KV side (also used for Q
172 |                 unless q_rope_* is provided).
173 |             rope_sin: Same shape as rope_cos.
174 |             q_rope_cos: (B, Lq, head_dim) or (1, Lq, head_dim), Q-specific
175 |                 RoPE for cross-attention with gathered positions.
176 |             q_rope_sin: Same shape as q_rope_cos.
177 |             need_weights: Compatibility parameter, not used.
178 |
179 |         Returns:
180 |             Tuple of (output, None).
181 |         """
182 |         B, Lq, _ = query.shape
183 |         Lk = key.shape[1]
184 |
185 |         # 1. Linear projection
186 |         Q = self.W_q(query)  # (B, Lq, D)
187 |         K = self.W_k(key)    # (B, Lk, D)
188 |         V = self.W_v(value)  # (B, Lk, D)
189 |
190 |         # 2. Reshape to (B, num_heads, L, head_dim)
191 |         Q = Q.view(B, Lq, self.num_heads, self.head_dim).transpose(1, 2)
```

```
        K = K.view(B, Lk, self.num_heads, self.head_dim).transpose(1, 2)
        V = V.view(B, Lk, self.num_heads, self.head_dim).transpose(1, 2)

        # 3. Apply RoPE independently to Q and K
        if rope_cos is not None and rope_sin is not None:
            # K always uses rope_cos/rope_sin (KV-side positional encoding)
            K = apply_rope_to_tensor(K, rope_cos, rope_sin)

            if self.rope_on_q:
                # Q side: prefer dedicated q_rope_cos/sin (top_k positions in LongerEncoder cross-attn)
                q_cos = q_rope_cos if q_rope_cos is not None else rope_cos
                q_sin = q_rope_sin if q_rope_sin is not None else rope_sin
                Q = apply_rope_to_tensor(Q, q_cos, q_sin)

        # 4. Convert key_padding_mask to SDPA format
        sdpa_attn_mask = None
        if key_padding_mask is not None:
            # key_padding_mask: (B, Lk), True = padding
            # SDPA expects (B, 1, 1, Lk) bool mask, True = attend
            sdpa_attn_mask = ~key_padding_mask.unsqueeze(1).unsqueeze(2)  # (B, 1, 1, Lk)
            sdpa_attn_mask = sdpa_attn_mask.expand(B, self.num_heads, Lq, Lk)

        if attn_mask is not None:
            # attn_mask: additive float mask (Lq, Lk), -inf means do not attend
            # Convert to bool: positions that are not -inf are True
            bool_attn = (attn_mask == 0)  # (Lq, Lk)
            bool_attn = bool_attn.unsqueeze(0).unsqueeze(0).expand(B, self.num_heads, Lq, Lk)
            if sdpa_attn_mask is not None:
                sdpa_attn_mask = sdpa_attn_mask & bool_attn
            else:
                sdpa_attn_mask = bool_attn

        # 5. Scaled Dot-Product Attention
        dropout_p = self.dropout if self.training else 0.0
        out = F.scaled_dot_product_attention(
            Q, K, V,
            attn_mask=sdpa_attn_mask,
            dropout_p=dropout_p,
        )  # (B, num_heads, Lq, head_dim)

        # Replace NaN from all-padding softmax with 0 (zero vectors preserve original input via residual)
        out = torch.nan_to_num(out, nan=0.0)

        # 6. Reshape back and output projection
        out = out.transpose(1, 2).contiguous().view(B, Lq, self.d_model)
        G = self.W_g(query)
        out = out * torch.sigmoid(G)
        out = self.W_o(out)

        return out, None

```

05. §5 CrossAttention

Lines 244-314 (71 lines) · Paper ref: HyFormer Query Decoding

飞机上读的时候重点关注：

- 和 self-attention 的唯一区别：Q 来源 vs K/V 来源不同
- HyFormer 中 Q = query tokens, K/V = sequence tokens

```
244 | class CrossAttention(nn.Module):
245 |     """Cross-attention module.
246 |
247 |     Query comes from global tokens (Q tokens), Key/Value comes from sequence
248 |     tokens. Only applies RoPE to KV side (rope_on_q=False).
249 |     """
250 |
251 |     def __init__(
252 |         self,
253 |         d_model: int,
254 |         num_heads: int,
255 |         dropout: float = 0.0,
256 |         ln_mode: str = 'pre'
257 |     ) -> None:
258 |         super().__init__()
259 |         self.ln_mode = ln_mode
260 |
261 |         self.attn = RoPEMultiheadAttention(
262 |             d_model=d_model,
263 |             num_heads=num_heads,
264 |             dropout=dropout,
265 |             rope_on_q=False,
266 |         )
267 |
268 |         if ln_mode in ['pre', 'post']:
269 |             self.norm_q = nn.LayerNorm(d_model)
270 |             self.norm_kv = nn.LayerNorm(d_model)
271 |
272 |     def forward(
273 |         self,
274 |         query: torch.Tensor,
275 |         key_value: torch.Tensor,
276 |         key_padding_mask: Optional[torch.Tensor] = None,
277 |         rope_cos: Optional[torch.Tensor] = None,
278 |         rope_sin: Optional[torch.Tensor] = None,
279 |     ) -> torch.Tensor:
280 |         """Computes cross-attention between query tokens and sequence tokens.
281 |
282 |         Args:
283 |             query: (B, Nq, D), query tokens.
284 |             key_value: (B, L, D), sequence tokens.
285 |             key_padding_mask: (B, L), True indicates padding positions.
286 |             rope_cos: (1, L, head_dim), KV-side RoPE cosine values.
287 |             rope_sin: (1, L, head_dim), KV-side RoPE sine values.
288 |
289 |         Returns:
290 |             Output tensor of shape (B, Nq, D).
291 |         """
292 |         residual = query
293 |
294 |         if self.ln_mode == 'pre':
295 |             query = self.norm_q(query)
296 |             key_value = self.norm_kv(key_value)
297 |
298 |         out, _ = self.attn(
299 |             query=query,
300 |             key=key_value,
301 |             value=key_value,
302 |             key_padding_mask=key_padding_mask,
303 |             rope_cos=rope_cos,
304 |             rope_sin=rope_sin,
305 |         )
306 |
307 |         out = residual + out
308 |
309 |         if self.ln_mode == 'post':
310 |             out = self.norm_q(out)
311 |
312 |         return out
313 |
314 |
```

06. §6 RankMixerBlock（Token Mixing 灵魂）

Lines 315-414 (100 lines) · Paper ref: 论文 4 RankMixer

飞机上读的时候重点关注：

- token_mixing: (B,T,D)→(B,T,T,d_sub)→transpose→(B,T,D) 无参数
- 仓库用 shared FFN (fc1/fc2 各一份)，论文原版是 Per-token FFN
- 手画 T=4, d_sub=2 的 reshape 过程验证理解

```
315 | class RankMixerBlock(nn.Module):
316 |     """HyFormer Query Boosting block.
317 |
318 |     Performs three steps:
319 |     1. Token Mixing: Parameter-free tensor reshaping.
320 |     2. Per-token FFN: Shared-parameter feedforward network.
321 |     3. Residual connection: Q_boost = Q + Q_e.
322 |
323 |     Constraint: d_model must be divisible by n_total in 'full' mode.
324 |     """
325 |
326 |     def __init__(
327 |         self,
328 |         d_model: int,
329 |         n_total: int,  # T = Nq + Nns
330 |         hidden_mult: int = 4,
331 |         dropout: float = 0.0,
332 |         mode: str = 'full'  # 'full' | 'ffn_only' | 'none'
333 |     ) -> None:
334 |         super().__init__()
335 |         self.T = n_total
336 |         self.D = d_model
337 |         self.mode = mode
338 |
339 |         if mode == 'none':
340 |             # Pure identity mapping, no submodules created
341 |             return
342 |
343 |         if mode == 'full':
344 |             if d_model % n_total != 0:
345 |                 raise ValueError(
346 |                     f"d_model={d_model} must be divisible by T={n_total} for token mixing."
347 |                 )
348 |             self.d_sub = d_model // n_total
349 |
350 |         # Per-token FFN (shared parameters) — used by both 'full' and 'ffn_only'
351 |         self.norm = nn.LayerNorm(d_model)
352 |         self.fc1 = nn.Linear(d_model, d_model * hidden_mult)
353 |         self.fc2 = nn.Linear(d_model * hidden_mult, d_model)
354 |         self.dropout = nn.Dropout(dropout)
355 |         # Post-LN after residual to stabilize stacked block outputs
356 |         self.post_norm = nn.LayerNorm(d_model)
357 |
358 |     def token_mixing(self, Q: torch.Tensor) -> torch.Tensor:
359 |         """Performs parameter-free token mixing via reshape and transpose.
360 |
361 |         Steps:
362 |         1. Splits channels into T subspaces: (B, T, D) -> (B, T, T, d_sub).
363 |         2. Swaps token and subspace axes: (B, token, h, d_sub) -> (B, h, token, d_sub).
364 |         3. Flattens back: (B, T, D).
365 |
366 |         Args:
367 |             Q: (B, T, D)
368 |
369 |         Returns:
370 |             Mixed tensor of shape (B, T, D).
371 |         """
372 |         B, T, D = Q.shape
373 |
374 |         # (B, T, D) -> (B, T, T, d_sub)
375 |         Q_split = Q.view(B, T, self.T, self.d_sub)
376 |
377 |         # (B, token, h, d_sub) -> (B, h, token, d_sub)
378 |         Q_rewired = Q_split.transpose(1, 2).contiguous()
379 |
380 |         # (B, T, T, d_sub) -> (B, T, D)
381 |         Q_hat = Q_rewired.view(B, T, D)
382 |         return Q_hat
383 |
384 |     def forward(self, Q: torch.Tensor) -> torch.Tensor:
385 |         """Applies query boosting: token mixing, FFN, and residual connection.
386 |
387 |         Args:
388 |             Q: (B, T, D) where T = Nq + Nns.
389 |
390 |         Returns:
391 |             Boosted tensor of shape (B, T, D).
```

```
        """
        if self.mode == 'none':
            return Q

        # Token Mixing (parameter-free rewire) or identity
        if self.mode == 'full':
            Q_hat = self.token_mixing(Q)
        else:  # 'ffn_only'
            Q_hat = Q

        # Per-token FFN
        x = self.norm(Q_hat)
        x = self.fc1(x)
        x = F.gelu(x)
        x = self.dropout(x)
        Q_e = self.fc2(x)

        # Residual from original Q
        Q_boost = Q + Q_e
        Q_boost = self.post_norm(Q_boost)
        return Q_boost


```

07. §7 MultiSeqQueryGenerator

Lines 415-557 (143 lines) · Paper ref: 论文 3 HyFormer §3.4 Multi-Sequence

飞机上读的时候重点关注：

- 每个 sequence 有 num_queries 个独立 FFN，生成 Q tokens
- GlobalInfo = Concat(NS 均池, Seq 均池)
- item_conditioned 分支是仓库 DIN-style 扩展（论文无）

```
415 | class TargetItemSeqInjector(nn.Module):
416 |     """Inject target-item representation into each sequence token (ADR-004).
417 |
418 |     Makes the per-sequence encoder target-aware by adding a per-sequence
419 |     projection of the target item representation to every seq token, before
420 |     the sequence encoder runs. This is orthogonal to the item-conditioned
421 |     query branch in :class:`MultiSeqQueryGenerator` (which acts on the query
422 |     side): injection here modifies the encoder's key/value input so the
423 |     encoder itself can perform target-specific matching.
424 |
425 |     Mode ``'additive'`` (default):
426 |         target_repr = mean(item_ns_tokens)            # (B, D)
427 |         target_proj_i = LinearPerSeq_i(target_repr)   # (B, D)
428 |         seq_tokens_i' = seq_tokens_i + α_i * target_proj_i.unsqueeze(1)
429 |
430 |     α_i is a learned per-sequence scalar initialised to ``alpha_init``
431 |     (default 0.0). With α=0 the injector is a functional no-op, so training
432 |     starts equivalent to the baseline and the optimiser decides the strength
433 |     of each domain's injection. This is the safest path to enable target-
434 |     aware conditioning without breaking a well-trained baseline.
435 |
436 |     Padding tokens still receive the injection at the token level, but the
437 |     downstream seq encoder masks them out via ``seq_padding_masks`` so they
438 |     contribute neither to pooling nor to attention output.
439 |
440 |     Mode ``'off'``: the module short-circuits (identity) and has no params.
441 |     """
442 |
443 |     def __init__(
444 |         self,
445 |         d_model: int,
446 |         num_sequences: int,
447 |         mode: str = 'off',
448 |         alpha_init: float = 0.0,
449 |     ) -> None:
450 |         super().__init__()
451 |         self.d_model = d_model
452 |         self.num_sequences = num_sequences
453 |         self.mode = mode
454 |         if mode == 'off':
455 |             return
456 |         if mode != 'additive':
457 |             raise ValueError(
458 |                 f"Unsupported target_item_seq_injection mode: {mode!r}. "
459 |                 f"Accepted: 'off', 'additive'.")
460 |         self.target_proj_per_seq = nn.ModuleList([
461 |             nn.Linear(d_model, d_model) for _ in range(num_sequences)
462 |         ])
463 |         # Per-sequence learnable scalar; init=0 → starts as a no-op so the
464 |         # optimiser decides how much each domain should attend to target.
465 |         self.alpha = nn.Parameter(
466 |             torch.full((num_sequences,), float(alpha_init)))
467 |
468 |     def forward(
469 |         self,
470 |         seq_tokens_list: list,
471 |         item_repr: torch.Tensor,
472 |     ) -> list:
473 |         """Add α_i * W_i(item_repr) to each token of sequence i.
474 |
475 |         Args:
476 |             seq_tokens_list: List of (B, L_i, D) tensors, length S.
477 |             item_repr: (B, D) target-item representation (mean of item NS
478 |                 tokens, same source as ICQ).
479 |
480 |         Returns:
481 |             New list of (B, L_i, D) tensors with target injected. Input list
482 |             is not mutated.
483 |         """
484 |         if self.mode == 'off':
485 |             return seq_tokens_list
486 |         out = []
487 |         for i in range(self.num_sequences):
488 |             # (B, D) → (B, 1, D), broadcast-add across sequence length
489 |             proj = self.target_proj_per_seq[i](item_repr).unsqueeze(1)
490 |             out.append(seq_tokens_list[i] + self.alpha[i] * proj)
491 |         return out
```

```
|
|
| class MultiSeqQueryGenerator(nn.Module):
|     """Multi-sequence query generation module.
|
|     Generates Q tokens independently for each sequence:
|     For each sequence i:
|         GlobalInfo_i = Concat(F1..FM, MeanPool(Seq_i))
|         Q_i = [FFN_{i,1}(GlobalInfo_i), ..., FFN_{i,N}(GlobalInfo_i)]
|
|     When ``item_conditioned=True``, a per-sequence learned gate blends the
|     standard global-info query with an item-aware branch:
|         item_repr   = mean(item_ns_tokens)        # (B, D)
|         gate_i      = σ(W_gate_i · [global_info_i, item_repr])  # (B, D)
|         q_{i,j}     = (1 - gate_i) ■ FFN_{i,j}(global_info_i)
|                     +       gate_i  ■ FFN_item_{i,j}(item_repr)
|     This lets each domain independently learn how much the target item
|     should steer its queries (DIN-style target-item awareness).
|     """
|
|     def __init__(
|         self,
|         d_model: int,
|         num_ns: int,
|         num_queries: int,
|         num_sequences: int,
|         hidden_mult: int = 4,
|         item_conditioned: bool = False,
|         num_item_ns: int = 0,
|     ) -> None:
|         super().__init__()
|         self.num_queries = num_queries
|         self.num_sequences = num_sequences
|         self.d_model = d_model
|         self.item_conditioned = item_conditioned
|         self.num_item_ns = num_item_ns
|
|         global_info_dim = (num_ns + 1) * d_model
|
|         # LayerNorm on global_info to prevent gradient explosion from large-dim concat
|         self.global_info_norm = nn.LayerNorm(global_info_dim)
|
|         # Each sequence has N independent FFNs
|         self.query_ffns_per_seq = nn.ModuleList([
|             nn.ModuleList([
|                 nn.Sequential(
|                     nn.Linear(global_info_dim, d_model * hidden_mult),
|                     nn.SiLU(),
|                     nn.Linear(d_model * hidden_mult, d_model),
|                     nn.LayerNorm(d_model),
|                 )
|                 for _ in range(num_queries)
|             ])
|             for _ in range(num_sequences)
|         ])
|
|         if item_conditioned and num_item_ns > 0:
|             # Item-aware branch: per-sequence, per-query FFN conditioned on item_repr
|             self.item_query_ffns_per_seq = nn.ModuleList([
|                 nn.ModuleList([
|                     nn.Sequential(
|                         nn.Linear(d_model, d_model * hidden_mult),
|                         nn.SiLU(),
|                         nn.Linear(d_model * hidden_mult, d_model),
|                         nn.LayerNorm(d_model),
|                     )
```

08. §8 SwiGLUEncoder (baseline 默认，3.5× 更快)

Lines 558-598 (41 lines) · Paper ref: —

飞机上读的时候重点关注：

- 为什么它比 Transformer 快 3.5×（attention-free）
- Pre-LN + SwiGLU + 残差的极简结构

```
558 |                     for _ in range(num_queries)
559 |                 ])
560 |                 for _ in range(num_sequences)
561 |             ])
562 |             # Per-sequence gate: [global_info, item_repr] → D-dim blend weight
563 |             self.item_gates = nn.ModuleList([
564 |                 nn.Sequential(
565 |                     nn.Linear(global_info_dim + d_model, d_model),
566 |                     nn.Sigmoid(),
567 |                 )
568 |                 for _ in range(num_sequences)
569 |             ])
570 |
571 |     def forward(
572 |         self,
573 |         ns_tokens: torch.Tensor,
574 |         seq_tokens_list: list,
575 |         seq_padding_masks: list
576 |     ) -> list:
577 |         """Generates query tokens for each sequence.
578 |
579 |         Args:
580 |             ns_tokens: (B, M, D), shared NS tokens.
581 |             seq_tokens_list: List of (B, L_i, D) tensors, length S.
582 |             seq_padding_masks: List of (B, L_i) masks, length S. True
583 |                 indicates padding.
584 |
585 |         Returns:
586 |             List of (B, Nq, D) query token tensors, length S.
587 |         """
588 |         B = ns_tokens.shape[0]
589 |         ns_flat = ns_tokens.view(B, -1)  # (B, M*D)
590 |
591 |         # Extract item representation for item-conditioned branch.
592 |         # item_ns tokens are the last num_item_ns tokens in ns_tokens.
593 |         item_repr: Optional[torch.Tensor] = None
594 |         if self.item_conditioned and self.num_item_ns > 0:
595 |             item_repr = ns_tokens[:, -self.num_item_ns:, :].mean(dim=1)  # (B, D)
596 |
597 |         q_tokens_list = []
598 |         for i in range(self.num_sequences):
```

09. §9 TransformerEncoder (对照)

Lines 600-671 (72 lines) · Paper ref: 论文 1 Attention

飞机上读的时候重点关注：

- 标准 Pre-LN Transformer encoder layer
- 对比 SwiGLUEncoder 理解 attention 带来的代价

```
600 |             valid_mask = ~seq_padding_masks[i]  # True = valid
601 |             valid_mask_expanded = valid_mask.unsqueeze(-1).float()  # (B, L_i, 1)
602 |             seq_sum = (seq_tokens_list[i] * valid_mask_expanded).sum(dim=1)  # (B, D)
603 |             seq_count = valid_mask_expanded.sum(dim=1).clamp(min=1)  # (B, 1)
604 |             seq_pooled = seq_sum / seq_count  # (B, D)
605 |
606 |             # GlobalInfo_i = Concat(NS_flat, seq_pooled_i)
607 |             global_info = torch.cat([ns_flat, seq_pooled], dim=-1)  # (B, (M+1)*D)
608 |             global_info = self.global_info_norm(global_info)
609 |
610 |             # Base queries from global info
611 |             base_queries = [ffn(global_info) for ffn in self.query_ffns_per_seq[i]]
612 |
613 |             if item_repr is not None:
614 |                 # Item-aware branch: gate blends base query with item-conditioned query
615 |                 gate = self.item_gates[i](
616 |                     torch.cat([global_info, item_repr], dim=-1))  # (B, D)
617 |                 item_queries = [
618 |                     ffn(item_repr) for ffn in self.item_query_ffns_per_seq[i]]
619 |                 queries = [
620 |                     (1.0 - gate) * bq + gate * iq
621 |                     for bq, iq in zip(base_queries, item_queries)
622 |                 ]
623 |             else:
624 |                 queries = base_queries
625 |
626 |             q_tokens = torch.stack(queries, dim=1)  # (B, Nq, D)
627 |             q_tokens_list.append(q_tokens)
628 |
629 |         return q_tokens_list
630 |
631 |
632 | # ■■■
```

10. §10 LongerEncoder

Lines 672-905 (234 lines) · Paper ref: 论文 7 LONGER

飞机上读的时候重点关注：

- L > top_k: Cross-Attn，Q = 最新 top_k，K/V = 全部
- L ≤ top_k: Self-Attn
- 复杂度 O(L²) → O(L_H · L_S)

```
672 |         x = self.norm(x)
673 |         x = self.swiglu(x)
674 |         x = self.dropout(x)
675 |         x = residual + x
676 |         return x, key_padding_mask
677 |
678 |
679 | class TransformerEncoder(nn.Module):
680 |     """High-capacity sequence encoder with self-attention and RoPE.
681 |
682 |     Structure: Standard Transformer Encoder Layer (Pre-LN).
683 |     """
684 |
685 |     def __init__(
686 |         self,
687 |         d_model: int,
688 |         num_heads: int,
689 |         hidden_mult: int = 4,
690 |         dropout: float = 0.0
691 |     ) -> None:
692 |         super().__init__()
693 |         self.norm1 = nn.LayerNorm(d_model)
694 |         self.norm2 = nn.LayerNorm(d_model)
695 |
696 |         self.self_attn = RoPEMultiheadAttention(
697 |             d_model=d_model,
698 |             num_heads=num_heads,
699 |             dropout=dropout,
700 |             rope_on_q=True,
701 |         )
702 |
703 |         hidden_dim = d_model * hidden_mult
704 |         self.ffn = nn.Sequential(
705 |             nn.Linear(d_model, hidden_dim),
706 |             nn.GELU(),
707 |             nn.Dropout(dropout),
708 |             nn.Linear(hidden_dim, d_model),
709 |             nn.Dropout(dropout)
710 |         )
711 |
712 |     def forward(
713 |         self,
714 |         x: torch.Tensor,
715 |         key_padding_mask: Optional[torch.Tensor] = None,
716 |         rope_cos: Optional[torch.Tensor] = None,
717 |         rope_sin: Optional[torch.Tensor] = None,
718 |     ) -> torch.Tensor:
719 |         """Applies one Transformer encoder layer.
720 |
721 |         Args:
722 |             x: (B, L, D)
723 |             key_padding_mask: (B, L), True indicates padding positions.
724 |             rope_cos: (1, L, head_dim), RoPE cosine values.
725 |             rope_sin: (1, L, head_dim), RoPE sine values.
726 |
727 |         Returns:
728 |             Tuple of (output tensor of shape (B, L, D), key_padding_mask).
729 |         """
730 |         # Self-Attention (Pre-LN) with RoPE
731 |         residual = x
732 |         x = self.norm1(x)
733 |         x, _ = self.self_attn(
734 |             query=x,
735 |             key=x,
736 |             value=x,
737 |             key_padding_mask=key_padding_mask,
738 |             rope_cos=rope_cos,
739 |             rope_sin=rope_sin,
740 |         )
741 |         x = residual + x
742 |
743 |         # FFN (Pre-LN)
744 |         residual = x
745 |         x = self.norm2(x)
746 |         x = self.ffn(x)
747 |         x = residual + x
748 |
```

```
        return x, key_padding_mask

class LongerEncoder(nn.Module):
    """Top-K compressed sequence encoder.

    Adapts behavior based on input length:
    - L > top_k (first MultiSeqHyFormerBlock): Cross Attention.
      Q = latest top_k tokens, K/V = all seq tokens -> output (B, top_k, D).
    - L <= top_k (subsequent MultiSeqHyFormerBlocks): Self Attention.
      Q = K = V = top_k tokens -> output (B, top_k, D).

    Causal mask is only applied among top_k tokens (self-attention layers);
    the first cross-attention layer does not use a causal mask since Q and K
    have different lengths.

    Returns (output, new_key_padding_mask) so downstream can update the mask.
    """

    def __init__(
        self,
        d_model: int,
        num_heads: int,
        top_k: int = 50,
        hidden_mult: int = 4,
        dropout: float = 0.0,
        causal: bool = False
    ) -> None:
        super().__init__()
        self.top_k = top_k
        self.causal = causal

        # Pre-LN for attention
        self.norm_q = nn.LayerNorm(d_model)
        self.norm_kv = nn.LayerNorm(d_model)

        # Shared RoPEMHA for both cross and self attention
        self.attn = RoPEMultiheadAttention(
            d_model=d_model,
            num_heads=num_heads,
            dropout=dropout,
            rope_on_q=True,
        )

        # FFN (Pre-LN + residual)
        self.ffn_norm = nn.LayerNorm(d_model)
        hidden_dim = d_model * hidden_mult
        self.ffn = nn.Sequential(
            nn.Linear(d_model, hidden_dim),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(hidden_dim, d_model),
            nn.Dropout(dropout)
        )

    def _gather_top_k(
        self,
        x: torch.Tensor,
        key_padding_mask: torch.Tensor
    ) -> Tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
        """Selects the latest top_k valid tokens from each sample.

        Args:
            x: (B, L, D)
            key_padding_mask: (B, L), True indicates padding.

        Returns:
            top_k_tokens: (B, top_k, D)
            new_padding_mask: (B, top_k), True indicates padding.
            position_indices: (B, top_k), original position index for each
                selected token, used for Q-side RoPE.
        """
        B, L, D = x.shape
        device = x.device

        # Valid lengths per sample
        valid_len = (~key_padding_mask).sum(dim=1)  # (B,)

        # Start position for each sample: max(valid_len - top_k, 0)
        actual_k = torch.clamp(valid_len, max=self.top_k)  # (B,)
        start_pos = valid_len - actual_k  # (B,)

        # Build gather indices: (B, top_k)
        offsets = torch.arange(self.top_k, device=device).unsqueeze(0).expand(B, -1)  # (B, top_k)
        indices = start_pos.unsqueeze(1) + offsets  # (B, top_k)

        # For samples with valid_len < top_k, early indices may exceed valid range;
        # clamp to [0, L-1] and handle via mask below
        indices = torch.clamp(indices, min=0, max=L - 1)

        # Gather: (B, top_k, D)
```

```
        indices_expanded = indices.unsqueeze(-1).expand(-1, -1, D)  # (B, top_k, D)
        top_k_tokens = torch.gather(x, dim=1, index=indices_expanded)

        # New padding mask: first (top_k - actual_k) positions are padding
        new_valid_len = actual_k  # (B,)
        pad_count = self.top_k - new_valid_len  # (B,)
        pos_indices = torch.arange(self.top_k, device=device).unsqueeze(0)  # (1, top_k)
        new_padding_mask = pos_indices < pad_count.unsqueeze(1)  # (B, top_k)

        # Zero out tokens at padding positions
        top_k_tokens = top_k_tokens * (~new_padding_mask).unsqueeze(-1).float()

        # position_indices for Q-side RoPE
        position_indices = indices  # (B, top_k)

        return top_k_tokens, new_padding_mask, position_indices

    def forward(
        self,
        x: torch.Tensor,
        key_padding_mask: Optional[torch.Tensor] = None,
        rope_cos: Optional[torch.Tensor] = None,
        rope_sin: Optional[torch.Tensor] = None,
    ) -> Tuple[torch.Tensor, torch.Tensor]:
        """Applies the LongerEncoder with adaptive cross/self attention.

        Args:
            x: (B, L, D), sequence tokens.
            key_padding_mask: (B, L), True indicates padding.
            rope_cos: (1, L, head_dim), RoPE cosine values (length must cover
                original sequence length L).
            rope_sin: (1, L, head_dim), RoPE sine values.

        Returns:
            output: (B, top_k, D), compressed sequence.
            new_key_padding_mask: (B, top_k), updated padding mask.
        """
        B, L, D = x.shape

        if L > self.top_k:
            # === Cross Attention mode (first MultiSeqHyFormerBlock) ===
            # 1. Extract latest top_k tokens as query
            q, new_mask, q_pos_indices = self._gather_top_k(x, key_padding_mask)

            # 2. Pre-LN
            q_normed = self.norm_q(q)
            kv_normed = self.norm_kv(x)

            # 3. Build Q-side RoPE cos/sin by gathering from global cos/sin at top_k positions
            q_rope_cos = None
            q_rope_sin = None
            if rope_cos is not None and rope_sin is not None:
                # rope_cos: (1, L_max, head_dim), q_pos_indices: (B, top_k)
                head_dim = rope_cos.shape[2]
                # Expand to batch dimension
                cos_expanded = rope_cos.expand(B, -1, -1)  # (B, L_max, head_dim)
                sin_expanded = rope_sin.expand(B, -1, -1)
                idx = q_pos_indices.unsqueeze(-1).expand(-1, -1, head_dim)  # (B, top_k, head_dim)
                q_rope_cos = torch.gather(cos_expanded, 1, idx)  # (B, top_k, head_dim)
                q_rope_sin = torch.gather(sin_expanded, 1, idx)

            # 4. Cross Attention (no causal mask since Q and K have different lengths)
            attn_out, _ = self.attn(
                query=q_normed,
                key=kv_normed,
                value=kv_normed,
                key_padding_mask=key_padding_mask,  # Original (B, L) mask
```

11. §11 MultiSeqHyFormerBlock（HyFormer 核心 block）

Lines 906-1043 (138 lines) · Paper ref: 论文 3 HyFormer §3.2-3.4

飞机上读的时候重点关注：

- Step 1 Sequence Evolve: 每个序列独立 encoder
- Step 2 Query Decode: 每个 Q 集合 cross-attend 到对应序列
- Step 3 Query Boost: 所有 Q + NS 拼起来过 RankMixer
- n_total = num_queries * S + num_ns，决定 d_model 约束

```
                rope_cos=rope_cos,
                rope_sin=rope_sin,
                q_rope_cos=q_rope_cos,
                q_rope_sin=q_rope_sin,
            )
            out = q + attn_out  # Residual based on q
        else:
            # === Self Attention mode (subsequent MultiSeqHyFormerBlocks) ===
            new_mask = key_padding_mask

            # Pre-LN (Q and KV share norm_q)
            x_normed = self.norm_q(x)

            # Causal mask
            attn_mask = None
            if self.causal:
                attn_mask = nn.Transformer.generate_square_subsequent_mask(
                    L, device=x.device
                )

            attn_out, _ = self.attn(
                query=x_normed,
                key=x_normed,
                value=x_normed,
                key_padding_mask=key_padding_mask,
                attn_mask=attn_mask,
                rope_cos=rope_cos,
                rope_sin=rope_sin,
            )
            out = x + attn_out

        # FFN (Pre-LN + residual)
        residual = out
        out = self.ffn_norm(out)
        out = self.ffn(out)
        out = residual + out

        return out, new_mask


def create_sequence_encoder(
    encoder_type: str,
    d_model: int,
    num_heads: int = 4,
    hidden_mult: int = 4,
    dropout: float = 0.0,
    top_k: int = 50,
    causal: bool = False
) -> nn.Module:
    """Creates a sequence encoder of the specified type.

    Args:
        encoder_type: One of 'swiglu', 'transformer', or 'longer'.
        d_model: Model dimension.
        num_heads: Number of attention heads (used by transformer/longer).
        hidden_mult: FFN expansion multiplier.
        dropout: Dropout rate.
        top_k: Compression length for LongerEncoder (only used by longer).
        causal: Whether to use causal mask in LongerEncoder (only used by
            longer).

    Returns:
        A sequence encoder module.
    """
    if encoder_type == 'swiglu':
        return SwiGLUEncoder(d_model, hidden_mult, dropout)
    elif encoder_type == 'transformer':
        return TransformerEncoder(d_model, num_heads, hidden_mult, dropout)
    elif encoder_type == 'longer':
        return LongerEncoder(d_model, num_heads, top_k, hidden_mult, dropout, causal)
    else:
        raise ValueError(f"Unknown encoder type: {encoder_type}")


# ■■■
```

```
# HyFormer Blocks
# ═══════════════════════════════════════════════════════════════════════════════

class MultiSeqHyFormerBlock(nn.Module):
    """Multi-sequence HyFormer block.

    Each of the S sequences independently performs Sequence Evolution and
    Query Decoding, then all Q tokens and shared NS tokens are merged for
    joint Query Boosting.
    """

    def __init__(
        self,
        d_model: int,
        num_heads: int,
        num_queries: int,
        num_ns: int,
        num_sequences: int,
        seq_encoder_type: str = 'swiglu',
        hidden_mult: int = 4,
        dropout: float = 0.0,
        top_k: int = 50,
        causal: bool = False,
        rank_mixer_mode: str = 'full'
    ) -> None:
        super().__init__()
        self.num_sequences = num_sequences
        self.num_queries = num_queries
        self.num_ns = num_ns

        # Independent sequence encoder per sequence
        self.seq_encoders = nn.ModuleList([
            create_sequence_encoder(
                encoder_type=seq_encoder_type,
                d_model=d_model,
                num_heads=num_heads,
                hidden_mult=hidden_mult,
                dropout=dropout,
                top_k=top_k,
                causal=causal
            )
            for _ in range(num_sequences)
        ])

        # Independent cross-attention per sequence
        self.cross_attns = nn.ModuleList([
            CrossAttention(
                d_model=d_model,
                num_heads=num_heads,
                dropout=dropout,
                ln_mode='pre'
            )
            for _ in range(num_sequences)
        ])

        # RankMixer: input token count = Nq * S + Nns
        n_total = num_queries * num_sequences + num_ns
        self.mixer = RankMixerBlock(
            d_model=d_model,
            n_total=n_total,
            hidden_mult=hidden_mult,
            dropout=dropout,
```

12. §12 OneTransMixedAttn（Mixed Parameterization）

Lines 1044-1217 (174 lines) · Paper ref: 论文 5 OneTrans §Mixed Attn

飞机上读的时候重点关注：

- W_Q_s/W_K_s/W_V_s/W_O_s: S-tokens 共享 4×d² 参数
- W_Q_ns[j]/.../W_O_ns[j]: 每个 NS token 独立 4×d²×M 参数
- 仓库 S-S 用 BIDIRECTIONAL 不用 causal（多 domain 按字母序拼，无时序）
- _build_causal_mixed_mask: 手画 L=4, M=3 的 7x7 mask 矩阵

```
1044 |             mode=rank_mixer_mode
1045 |         )
1046 |
1047 |     def forward(
1048 |         self,
1049 |         q_tokens_list: list,
1050 |         ns_tokens: torch.Tensor,
1051 |         seq_tokens_list: list,
1052 |         seq_padding_masks: list,
1053 |         rope_cos_list: Optional[List[torch.Tensor]] = None,
1054 |         rope_sin_list: Optional[List[torch.Tensor]] = None,
1055 |     ) -> Tuple[list, torch.Tensor, list, list]:
1056 |         """Processes one multi-sequence HyFormer block step.
1057 |
1058 |         Args:
1059 |             q_tokens_list: List of (B, Nq, D) tensors, length S.
1060 |             ns_tokens: (B, Nns, D)
1061 |             seq_tokens_list: List of (B, L_i, D) tensors, length S.
1062 |             seq_padding_masks: List of (B, L_i) masks, length S.
1063 |             rope_cos_list: List of (1, L_i, head_dim) tensors, length S.
1064 |             rope_sin_list: List of (1, L_i, head_dim) tensors, length S.
1065 |
1066 |         Returns:
1067 |             A tuple (next_q_list, next_ns, next_seq_list, next_masks), where
1068 |             next_q_list is a list of (B, Nq, D) updated query tensors,
1069 |             next_ns is (B, Nns, D) updated non-sequence tokens,
1070 |             next_seq_list is a list of (B, L_i', D) encoded sequence tensors,
1071 |             and next_masks is a list of (B, L_i') updated padding masks.
1072 |         """
1073 |         S = self.num_sequences
1074 |         Nq = self.num_queries
1075 |
1076 |         # 1. Independent Sequence Evolution per sequence
1077 |         next_seqs = []
1078 |         next_masks = []
1079 |         for i in range(S):
1080 |             rc = rope_cos_list[i] if rope_cos_list is not None else None
1081 |             rs = rope_sin_list[i] if rope_sin_list is not None else None
1082 |             result = self.seq_encoders[i](
1083 |                 seq_tokens_list[i], seq_padding_masks[i],
1084 |                 rope_cos=rc, rope_sin=rs,
1085 |             )
1086 |             next_seq_i, mask_i = result
1087 |             next_seqs.append(next_seq_i)
1088 |             next_masks.append(mask_i)
1089 |
1090 |         # 2. Independent Query Decoding per sequence
1091 |         decoded_qs = []
1092 |         for i in range(S):
1093 |             rc = rope_cos_list[i] if rope_cos_list is not None else None
1094 |             rs = rope_sin_list[i] if rope_sin_list is not None else None
1095 |             decoded_q_i = self.cross_attns[i](
1096 |                 q_tokens_list[i], next_seqs[i], next_masks[i],
1097 |                 rope_cos=rc, rope_sin=rs,
1098 |             )
1099 |             decoded_qs.append(decoded_q_i)
1100 |
1101 |         # 3. Token Fusion: concatenate all decoded_q + ns_tokens
1102 |         combined = torch.cat(decoded_qs + [ns_tokens], dim=1)  # (B, Nq*S + Nns, D)
1103 |
1104 |         # 4. Query Boosting
1105 |         boosted = self.mixer(combined)  # (B, Nq*S + Nns, D)
1106 |
1107 |         # 5. Split back into per-sequence Q and NS
1108 |         next_q_list = []
1109 |         offset = 0
1110 |         for i in range(S):
1111 |             next_q_list.append(boosted[:, offset:offset + Nq, :])
1112 |             offset += Nq
1113 |         next_ns = boosted[:, offset:, :]
1114 |
1115 |         return next_q_list, next_ns, next_seqs, next_masks
1116 |
1117 |
1118 | # ■■■
```

```
# OneTrans: Mixed-Parameterization Unified Attention Block
# ████████████████████████████████████████████████████████████████████████████████████████████


class OneTransMixedAttn(nn.Module):
    """Mixed-Parameterization Multi-Head Attention for OneTrans.

    Sequence layout: [S_token_0 ... S_token_{L-1} | NS_token_0 ... NS_token_{M-1}]

    Attention mask (True = can attend):
    - S_i  → S_0..S_i (causal), S cannot see NS
    - NS_j → all S (full), NS_0..NS_j (causal within NS)

    Mixed parameterization:
    - S positions: shared W_Q^s, W_K^s, W_V^s, W_O^s
    - Each NS_j:   independent W_Q^j, W_K^j, W_V^j, W_O^j
    """

    def __init__(
        self,
        d_model: int,
        num_heads: int,
        num_ns: int,
        dropout: float = 0.0,
    ) -> None:
        super().__init__()
        assert d_model % num_heads == 0
        self.d_model = d_model
        self.num_heads = num_heads
        self.head_dim = d_model // num_heads
        self.num_ns = num_ns
        self.dropout = dropout

        # Shared projections for S-tokens
        self.W_Q_s = nn.Linear(d_model, d_model, bias=False)
        self.W_K_s = nn.Linear(d_model, d_model, bias=False)
        self.W_V_s = nn.Linear(d_model, d_model, bias=False)
        self.W_O_s = nn.Linear(d_model, d_model)

        # Per-NS-token independent projections
        self.W_Q_ns = nn.ModuleList(
            [nn.Linear(d_model, d_model, bias=False) for _ in range(num_ns)])
        self.W_K_ns = nn.ModuleList(
            [nn.Linear(d_model, d_model, bias=False) for _ in range(num_ns)])
        self.W_V_ns = nn.ModuleList(
            [nn.Linear(d_model, d_model, bias=False) for _ in range(num_ns)])
        self.W_O_ns = nn.ModuleList(
            [nn.Linear(d_model, d_model) for _ in range(num_ns)])

        # Gating for S output (same pattern as RoPEMultiheadAttention)
        self.W_g_s = nn.Linear(d_model, d_model)
        nn.init.zeros_(self.W_g_s.weight)
        nn.init.constant_(self.W_g_s.bias, 1.0)

    def _project_ns(
        self,
        ns: torch.Tensor,
        proj_list: nn.ModuleList,
    ) -> torch.Tensor:
        """Apply per-token linear to each NS token independently."""
        # ns: (B, M, D) → (B, M, D)
        parts = [proj_list[j](ns[:, j, :]).unsqueeze(1) for j in range(self.num_ns)]
        return torch.cat(parts, dim=1)  # (B, M, D)

    @staticmethod
    def _build_causal_mixed_mask(
        L: int,
        M: int,
        device: torch.device,
    ) -> torch.Tensor:
        """Build a bool mask (L+M, L+M) where True = allowed to attend.

        Pattern:
          [ones_LxL     |  zeros_LxM  ]
          [ones_MxL     | causal_MxM  ]

        S tokens use BIDIRECTIONAL attention among themselves (all S can
        see all other S). Cross-domain causality would be arbitrary because
        S tokens from different domains are concatenated in alphabet order,
        not temporal order. NS tokens attend to all S tokens and causally
        to previous NS tokens.
        """
        total = L + M
        mask = torch.zeros(total, total, dtype=torch.bool, device=device)
        # S-S block: bidirectional (every S sees every S)
        if L > 0:
            mask[:L, :L] = True
        # NS-S block: every NS sees all S
        if M > 0 and L > 0:
            mask[L:, :L] = True
```

```
        # NS-NS block: causal (NS_j sees NS_0 ... NS_j)
        if M > 0:
            mask[L:, L:] = torch.tril(
                torch.ones(M, M, dtype=torch.bool, device=device))
        return mask  # (L+M, L+M)

    def forward(
        self,
        s_tokens: torch.Tensor,
```

13. §13 OneTransMixedFFN

Lines 1218-1280 (63 lines) · Paper ref: 论文 5 OneTrans §Mixed FFN

飞机上读的时候重点关注：

- S 共享 FFN / 每个 NS 独立 FFN
- 参数量: FFN_s (1 份) + FFN_ns[j] (M 份)

```
1218 |         ns_tokens: torch.Tensor,
1219 |         s_padding_mask: Optional[torch.Tensor] = None,
1220 |     ) -> Tuple[torch.Tensor, torch.Tensor]:
1221 |         """Mixed-param attention over the unified [S | NS] sequence.
1222 |
1223 |         Args:
1224 |             s_tokens: (B, L, D)  – all domain sequence tokens, concatenated.
1225 |             ns_tokens: (B, M, D) – NS tokens.
1226 |             s_padding_mask: (B, L) bool, True = padding (masked out).
1227 |
1228 |         Returns:
1229 |             s_out: (B, L, D)
1230 |             ns_out: (B, M, D)
1231 |         """
1232 |         B, L, D = s_tokens.shape
1233 |         M = self.num_ns
1234 |
1235 |         # ---- QKV projections ----
1236 |         Q_s = self.W_Q_s(s_tokens)            # (B, L, D)
1237 |         K_s = self.W_K_s(s_tokens)
1238 |         V_s = self.W_V_s(s_tokens)
1239 |
1240 |         Q_ns = self._project_ns(ns_tokens, self.W_Q_ns)   # (B, M, D)
1241 |         K_ns = self._project_ns(ns_tokens, self.W_K_ns)
1242 |         V_ns = self._project_ns(ns_tokens, self.W_V_ns)
1243 |
1244 |         # Unified sequences
1245 |         Q = torch.cat([Q_s, Q_ns], dim=1)     # (B, L+M, D)
1246 |         K = torch.cat([K_s, K_ns], dim=1)
1247 |         V = torch.cat([V_s, V_ns], dim=1)
1248 |
1249 |         # ---- Reshape for multi-head ----
1250 |         def _to_heads(x: torch.Tensor) -> torch.Tensor:
1251 |             Bx, Lx, _ = x.shape
1252 |             return x.view(Bx, Lx, self.num_heads, self.head_dim).transpose(1, 2)
1253 |
1254 |         Q = _to_heads(Q)  # (B, H, L+M, head_dim)
1255 |         K = _to_heads(K)
1256 |         V = _to_heads(V)
1257 |
1258 |         # ---- Attention mask ----
1259 |         causal_mixed = self._build_causal_mixed_mask(L, M, s_tokens.device)
1260 |         # Expand to (B, 1, L+M, L+M)
1261 |         attn_mask = causal_mixed.unsqueeze(0).unsqueeze(0)
1262 |
1263 |         # Incorporate S padding: padded S positions cannot be attended to.
1264 |         if s_padding_mask is not None:
1265 |             # s_padding_mask: (B, L), True = pad → should NOT be attended to
1266 |             # Convert to (B, 1, 1, L+M): False where allowed
1267 |             pad_ext = torch.zeros(
1268 |                 B, 1, 1, L + M, dtype=torch.bool, device=s_tokens.device)
1269 |             pad_ext[:, :, :, :L] = s_padding_mask.unsqueeze(1).unsqueeze(2)
1270 |             # Combined: allowed = causal_mixed AND NOT padded
1271 |             attn_mask = attn_mask & (~pad_ext)
1272 |
1273 |         dp = self.dropout if self.training else 0.0
1274 |         out = F.scaled_dot_product_attention(
1275 |             Q, K, V, attn_mask=attn_mask, dropout_p=dp)
1276 |         out = torch.nan_to_num(out, nan=0.0)
1277 |         # (B, H, L+M, head_dim) → (B, L+M, D)
1278 |         out = out.transpose(1, 2).contiguous().view(B, L + M, D)
1279 |
1280 |         # ---- Split back + mixed output projection ----
```

14. §14 OneTransNSConditioner（EXP-013 +0.010 AUC 源头）

Lines 1281-1349 (69 lines) · Paper ref: 仓库自研扩展（有创新奖潜质）

飞机上读的时候重点关注：

- 在 OneTrans 主 block 前，给 NS token 加序列 mean-pool 信息
- 每个 NS 独立 conditioning FFN（mixed-param 一致）
- v2 无此模块: 0.848712 full train；v3 有: 0.859408 (+0.010)
- 飞机上想清楚：为什么这个能带来 +0.010？类比 HyFormer 的 MultiSeqQueryGenerator（都是给跨特征交互提供 warm-start）

```
1281 |         out_s_raw = out[:, :L, :]   # (B, L, D)
1282 |         out_ns_raw = out[:, L:, :]  # (B, M, D)
1283 |
1284 |         # S output: shared W_O_s with gating
1285 |         G = torch.sigmoid(self.W_g_s(s_tokens))
1286 |         s_out = self.W_O_s(out_s_raw) * G  # (B, L, D)
1287 |
1288 |         # NS output: per-token W_O_ns
1289 |         ns_parts = [
1290 |             self.W_O_ns[j](out_ns_raw[:, j, :]).unsqueeze(1) for j in range(M)
1291 |         ]
1292 |         ns_out = torch.cat(ns_parts, dim=1)  # (B, M, D)
1293 |
1294 |         return s_out, ns_out
1295 |
1296 |
1297 | class OneTransMixedFFN(nn.Module):
1298 |     """Mixed-Parameterization Feed-Forward Network.
1299 |
1300 |     S-tokens: shared W1, W2.
1301 |     Each NS_j: independent W1_j, W2_j.
1302 |     """
1303 |
1304 |     def __init__(
1305 |         self,
1306 |         d_model: int,
1307 |         num_ns: int,
1308 |         hidden_mult: int = 4,
1309 |         dropout: float = 0.0,
1310 |     ) -> None:
1311 |         super().__init__()
1312 |         hidden = d_model * hidden_mult
1313 |         self.num_ns = num_ns
1314 |
1315 |         # Shared FFN for S tokens
1316 |         self.ffn_s = nn.Sequential(
1317 |             nn.Linear(d_model, hidden),
1318 |             nn.GELU(),
1319 |             nn.Dropout(dropout),
1320 |             nn.Linear(hidden, d_model),
1321 |         )
1322 |
1323 |         # Per-NS-token independent FFN
1324 |         self.ffn_ns = nn.ModuleList([
1325 |             nn.Sequential(
1326 |                 nn.Linear(d_model, hidden),
1327 |                 nn.GELU(),
1328 |                 nn.Dropout(dropout),
1329 |                 nn.Linear(hidden, d_model),
1330 |             )
1331 |             for _ in range(num_ns)
1332 |         ])
1333 |
1334 |     def forward(
1335 |         self,
1336 |         s_tokens: torch.Tensor,
1337 |         ns_tokens: torch.Tensor,
1338 |     ) -> Tuple[torch.Tensor, torch.Tensor]:
1339 |         """Apply mixed FFN independently to S and NS tokens.
1340 |
1341 |         Args:
1342 |             s_tokens: (B, L, D)
1343 |             ns_tokens: (B, M, D)
1344 |
1345 |         Returns:
1346 |             s_out: (B, L, D)
1347 |             ns_out: (B, M, D)
1348 |         """
1349 |         s_out = self.ffn_s(s_tokens)
```

15. §15 OneTransBlock

Lines 1350-1484 (135 lines) · Paper ref: 论文 5 OneTrans (整体装配)

飞机上读的时候重点关注：

- 整体对比 MultiSeqHyFormerBlock 的三步流程
- OneTrans: S+NS 拼成一个 attention 矩阵 + mixed param
- HyFormer: cross-attn 桥接 + RankMixer boost

```
1350 |
1351 |         ns_parts = [
1352 |             self.ffn_ns[j](ns_tokens[:, j, :]).unsqueeze(1)
1353 |             for j in range(self.num_ns)
1354 |         ]
1355 |         ns_out = torch.cat(ns_parts, dim=1)  # (B, M, D)
1356 |
1357 |         return s_out, ns_out
1358 |
1359 |
1360 | class OneTransNSConditioner(nn.Module):
1361 |     """Sequence-conditioned initialization for OneTrans NS tokens.
1362 |
1363 |     Before the unified attention starts, each NS token receives an additive
1364 |     update from the mean-pooled domain sequence representations. This gives
1365 |     NS tokens initial sequence context analogous to HyFormer's
1366 |     MultiSeqQueryGenerator, preventing cold-start of NS → S attention.
1367 |
1368 |     Each NS token has an independent conditioning FFN (mixed-parameterization
1369 |     consistent with OneTrans design).
1370 |     """
1371 |
1372 |     def __init__(
1373 |         self,
1374 |         d_model: int,
1375 |         num_ns: int,
1376 |         num_sequences: int,
1377 |         hidden_mult: int = 2,
1378 |     ) -> None:
1379 |         super().__init__()
1380 |         self.num_ns = num_ns
1381 |         seq_dim = num_sequences * d_model
1382 |         # Pre-LN on the concatenated sequence context
1383 |         self.seq_norm = nn.LayerNorm(seq_dim)
1384 |         # Per-NS-token independent conditioning FFN
1385 |         self.conditioners = nn.ModuleList([
1386 |             nn.Sequential(
1387 |                 nn.Linear(seq_dim, d_model * hidden_mult),
1388 |                 nn.SiLU(),
1389 |                 nn.Linear(d_model * hidden_mult, d_model),
1390 |                 nn.LayerNorm(d_model),
1391 |             )
1392 |             for _ in range(num_ns)
1393 |         ])
1394 |
1395 |     def forward(
1396 |         self,
1397 |         ns_tokens: torch.Tensor,
1398 |         compressed_s_list: list,
1399 |         compressed_mask_list: list,
1400 |     ) -> torch.Tensor:
1401 |         """Add sequence-conditioned delta to each NS token.
1402 |
1403 |         Args:
1404 |             ns_tokens: (B, M, D)
1405 |             compressed_s_list: list of (B, K, D), one per domain.
1406 |             compressed_mask_list: list of (B, K) bool, True = padding.
1407 |
1408 |         Returns:
1409 |             ns_tokens updated: (B, M, D)
1410 |         """
1411 |         # Masked mean-pool each domain's compressed S tokens → (B, D) per domain
1412 |         domain_means = []
1413 |         for s_i, mask_i in zip(compressed_s_list, compressed_mask_list):
1414 |             valid = (~mask_i).float().unsqueeze(-1)      # (B, K, 1)
1415 |             s_mean = (s_i * valid).sum(dim=1) / valid.sum(dim=1).clamp(min=1)
1416 |             domain_means.append(s_mean)
1417 |
1418 |         seq_ctx = torch.cat(domain_means, dim=-1)        # (B, num_seq * D)
1419 |         seq_ctx = self.seq_norm(seq_ctx)
1420 |
1421 |         # Per-NS-token update
1422 |         updates = [self.conditioners[j](seq_ctx).unsqueeze(1)
1423 |                    for j in range(self.num_ns)]
1424 |         delta = torch.cat(updates, dim=1)                # (B, M, D)
1425 |
1426 |         return ns_tokens + delta
```

```
|
|
| class OneTransBlock(nn.Module):
|     """Single OneTrans layer (Pre-LN).
|
|     Layout: [compressed_S_tokens | NS_tokens]
|     - Mixed-param attention (bidirectional S, NS sees all S + causal NS)
|     - Mixed-param FFN (shared for S, independent per NS)
|     - Residual connections + LayerNorm
|
|     The sequence compression (top-K selection per domain) is handled
|     externally by ``_gather_onetrans_s_tokens`` in ``PCVRHyFormer``.
|     """
|
|     def __init__(
|         self,
|         d_model: int,
|         num_heads: int,
|         num_ns: int,
|         hidden_mult: int = 4,
|         dropout: float = 0.0,
|     ) -> None:
|         super().__init__()
|         self.norm_attn_s = nn.LayerNorm(d_model)
|         self.norm_attn_ns = nn.LayerNorm(d_model)
|         self.attn = OneTransMixedAttn(d_model, num_heads, num_ns, dropout)
|
|         self.norm_ffn_s = nn.LayerNorm(d_model)
|         self.norm_ffn_ns = nn.LayerNorm(d_model)
|         self.ffn = OneTransMixedFFN(d_model, num_ns, hidden_mult, dropout)
|
|     def forward(
|         self,
|         s_tokens: torch.Tensor,
|         ns_tokens: torch.Tensor,
|         s_padding_mask: Optional[torch.Tensor] = None,
|     ) -> Tuple[torch.Tensor, torch.Tensor]:
|         """Apply one OneTrans layer with Pre-LN and residuals.
|
|         Args:
|             s_tokens: (B, L, D) concatenated sequence tokens from all domains.
|             ns_tokens: (B, M, D) NS tokens (user + item).
|             s_padding_mask: (B, L) bool, True = padding.
|
|         Returns:
|             s_tokens: (B, L, D) updated
|             ns_tokens: (B, M, D) updated
|         """
|         # Pre-LN attention
|         s_normed = self.norm_attn_s(s_tokens)
|         ns_normed = self.norm_attn_ns(ns_tokens)
|         s_attn, ns_attn = self.attn(s_normed, ns_normed, s_padding_mask)
|
|         s_tokens = s_tokens + s_attn
|         ns_tokens = ns_tokens + ns_attn
|
|         # Pre-LN FFN
|         s_normed = self.norm_ffn_s(s_tokens)
```

16. §16 GroupNSTokenizer (原 baseline 方式)

Lines 1485-1566 (82 lines) · Paper ref: —

飞机上读的时候重点关注：

- 怎么把 sparse features 按 group 聚成 NS tokens

```
1485 |         ns_normed = self.norm_ffn_ns(ns_tokens)
1486 |         s_ffn, ns_ffn = self.ffn(s_normed, ns_normed)
1487 |
1488 |         s_tokens = s_tokens + s_ffn
1489 |         ns_tokens = ns_tokens + ns_ffn
1490 |
1491 |         return s_tokens, ns_tokens
1492 |
1493 |
1494 |
1495 |     """Single OneTrans layer (Pre-LN).
1496 |
1497 |     Layout: [compressed_S_tokens | NS_tokens]
1498 |     - Mixed-param attention (causal S, NS sees all S + causal NS)
1499 |     - Mixed-param FFN (shared for S, independent per NS)
1500 |     - Residual connections + LayerNorm
1501 |
1502 |     The sequence compression (top-K selection per domain) is handled
1503 |     externally by ``_gather_onetrans_s_tokens`` in ``PCVRHyFormer``.
1504 |     """
1505 |
1506 |     def __init__(
1507 |         self,
1508 |         d_model: int,
1509 |         num_heads: int,
1510 |         num_ns: int,
1511 |         hidden_mult: int = 4,
1512 |         dropout: float = 0.0,
1513 |     ) -> None:
1514 |         super().__init__()
1515 |         self.norm_attn_s = nn.LayerNorm(d_model)
1516 |         self.norm_attn_ns = nn.LayerNorm(d_model)
1517 |         self.attn = OneTransMixedAttn(d_model, num_heads, num_ns, dropout)
1518 |
1519 |         self.norm_ffn_s = nn.LayerNorm(d_model)
1520 |         self.norm_ffn_ns = nn.LayerNorm(d_model)
1521 |         self.ffn = OneTransMixedFFN(d_model, num_ns, hidden_mult, dropout)
1522 |
1523 |     def forward(
1524 |         self,
1525 |         s_tokens: torch.Tensor,
1526 |         ns_tokens: torch.Tensor,
1527 |         s_padding_mask: Optional[torch.Tensor] = None,
1528 |     ) -> Tuple[torch.Tensor, torch.Tensor]:
1529 |         """Apply one OneTrans layer with Pre-LN and residuals.
1530 |
1531 |         Args:
1532 |             s_tokens: (B, L, D) concatenated sequence tokens from all domains.
1533 |             ns_tokens: (B, M, D) NS tokens (user + item).
1534 |             s_padding_mask: (B, L) bool, True = padding.
1535 |
1536 |         Returns:
1537 |             s_tokens: (B, L, D) updated
1538 |             ns_tokens: (B, M, D) updated
1539 |         """
1540 |         # Pre-LN attention
1541 |         s_normed = self.norm_attn_s(s_tokens)
1542 |         ns_normed = self.norm_attn_ns(ns_tokens)
1543 |         s_attn, ns_attn = self.attn(s_normed, ns_normed, s_padding_mask)
1544 |
1545 |         s_tokens = s_tokens + s_attn
1546 |         ns_tokens = ns_tokens + ns_attn
1547 |
1548 |         # Pre-LN FFN
1549 |         s_normed = self.norm_ffn_s(s_tokens)
1550 |         ns_normed = self.norm_ffn_ns(ns_tokens)
1551 |         s_ffn, ns_ffn = self.ffn(s_normed, ns_normed)
1552 |
1553 |         s_tokens = s_tokens + s_ffn
1554 |         ns_tokens = ns_tokens + ns_ffn
1555 |
1556 |         return s_tokens, ns_tokens
1557 |
1558 |
1559 | # ================================================================================
1560 | # PCVRHyFormer Main Model
1561 | # ================================================================================
1562 |
1563 |
1564 | class GroupNSTokenizer(nn.Module):
1565 |     """NS tokenizer used by ns_tokenizer_type='group'.
```

1566 |

17. §17 RankMixerNSTokenizer (当前默认)

Lines 1567-1688 (122 lines) · Paper ref: 论文 4 RankMixer NS 部分

飞机上读的时候重点关注：

- 为什么当前默认选它而非 Group
- user_ns_tokens=5, item_ns_tokens=2 的设计动机

```
1567 |     Groups discrete features by fid, applies shared embedding with mean
1568 |     pooling per multi-valued feature, then projects each group to a single
1569 |     NS token (one token per group).
1570 |     """
1571 |
1572 |     def __init__(self, feature_specs: List[Tuple[int, int, int]],
1573 |                  groups: List[List[int]], emb_dim: int, d_model: int,
1574 |                  emb_skip_threshold: int = 0) -> None:
1575 |         super().__init__()
1576 |         self.feature_specs = feature_specs
1577 |         self.groups = groups
1578 |         self.emb_dim = emb_dim
1579 |         self.emb_skip_threshold = emb_skip_threshold
1580 |
1581 |         # One embedding table per fid (None if skipped by emb_skip_threshold
1582 |         # or if vocab_size <= 0 / no vocab info).
1583 |         embs = []
1584 |         for vs, offset, length in feature_specs:
1585 |             skip = int(vs) <= 0 or (emb_skip_threshold > 0 and int(vs) > emb_skip_threshold)
1586 |             if skip:
1587 |                 embs.append(None)
1588 |             else:
1589 |                 embs.append(nn.Embedding(int(vs) + 1, emb_dim, padding_idx=0))
1590 |         self.embs = nn.ModuleList([e for e in embs if e is not None])
1591 |         # Map from fid index to position in self.embs (or -1 if filtered)
1592 |         self._emb_index = []
1593 |         real_idx = 0
1594 |         for e in embs:
1595 |             if e is not None:
1596 |                 self._emb_index.append(real_idx)
1597 |                 real_idx += 1
1598 |             else:
1599 |                 self._emb_index.append(-1)
1600 |
1601 |         # Per-group projection: num_fids_in_group * emb_dim -> d_model (with LayerNorm)
1602 |         self.group_projs = nn.ModuleList([
1603 |             nn.Sequential(
1604 |                 nn.Linear(len(group) * emb_dim, d_model),
1605 |                 nn.LayerNorm(d_model),
1606 |             )
1607 |             for group in groups
1608 |         ])
1609 |
1610 |     def forward(self, int_feats: torch.Tensor) -> torch.Tensor:
1611 |         """Embeds and projects grouped discrete features into NS tokens.
1612 |
1613 |         Args:
1614 |             int_feats: (B, total_int_dim), concatenated integer features.
1615 |
1616 |         Returns:
1617 |             Tokens of shape (B, num_groups, D).
1618 |         """
1619 |         tokens = []
1620 |         for group, proj in zip(self.groups, self.group_projs):
1621 |             fid_embs = []
1622 |             for fid_idx in group:
1623 |                 vs, offset, length = self.feature_specs[fid_idx]
1624 |                 emb_real_idx = self._emb_index[fid_idx]
1625 |                 if emb_real_idx == -1:
1626 |                     # Filtered high-cardinality feature: output zero vector
1627 |                     fid_emb = int_feats.new_zeros(int_feats.shape[0], self.emb_dim)
1628 |                 else:
1629 |                     emb_layer = self.embs[emb_real_idx]
1630 |                     if length == 1:
1631 |                         # Single-value feature: direct lookup
1632 |                         fid_emb = emb_layer(int_feats[:, offset].long())  # (B, emb_dim)
1633 |                     else:
1634 |                         # Multi-value feature: lookup then mean pooling (ignoring padding=0)
1635 |                         vals = int_feats[:, offset:offset + length].long()  # (B, length)
1636 |                         emb_all = emb_layer(vals)  # (B, length, emb_dim)
1637 |                         mask = (vals != 0).float().unsqueeze(-1)  # (B, length, 1)
1638 |                         count = mask.sum(dim=1).clamp(min=1)  # (B, 1)
1639 |                         fid_emb = (emb_all * mask).sum(dim=1) / count  # (B, emb_dim)
1640 |                 fid_embs.append(fid_emb)
1641 |             cat_emb = torch.cat(fid_embs, dim=-1)  # (B, num_fids*emb_dim)
1642 |             tokens.append(F.silu(proj(cat_emb)).unsqueeze(1))  # (B, 1, D)
1643 |         return torch.cat(tokens, dim=1)  # (B, num_groups, D)
1644 |
1645 |
```

```
class RankMixerNSTokenizer(nn.Module):
    """NS Tokenizer following the RankMixer paper's approach.

    All group embedding vectors are concatenated into a single long vector,
    then equally split into num_ns_tokens segments, each projected to d_model.
    This allows num_ns_tokens to be chosen freely (independent of group count).
    """

    def __init__(
        self,
        feature_specs: List[Tuple[int, int, int]],
        groups: List[List[int]],
        emb_dim: int,
        d_model: int,
        num_ns_tokens: int,
        emb_skip_threshold: int = 0,
    ) -> None:
        """Initializes RankMixerNSTokenizer.

        Args:
            feature_specs: [(vocab_size, offset, length), ...] per feature.
            groups: List of feature index groups (defines semantic ordering).
            emb_dim: Embedding dimension per feature.
            d_model: Output token dimension.
            num_ns_tokens: Number of NS tokens to produce (T segments).
            emb_skip_threshold: Skip embedding for features with vocab > threshold.
        """
        super().__init__()
        self.feature_specs = feature_specs
        self.groups = groups
        self.emb_dim = emb_dim
        self.num_ns_tokens = num_ns_tokens
        self.emb_skip_threshold = emb_skip_threshold

        # One embedding table per fid (None if skipped by emb_skip_threshold
        # or if vocab_size <= 0 / no vocab info).
        embs = []
        for vs, offset, length in feature_specs:
            skip = int(vs) <= 0 or (emb_skip_threshold > 0 and int(vs) > emb_skip_threshold)
            if skip:
                embs.append(None)
            else:
                embs.append(nn.Embedding(int(vs) + 1, emb_dim, padding_idx=0))
```

18. §18 PCVRHyFormer（顶层装配）

Lines 1689-2396 (708 lines) · Paper ref: 整体装配（所有论文的综合体现）

飞机上读的时候重点关注：

- 先过一遍数据流：输入 → NS tokenize → Query gen → blocks → 分类
- 理解 use_onetrans 开关切换 HyFormer vs OneTrans 的位置
- enable_dense_bypass / item_conditioned_query / seq_hash_vocab 扩展参数

```
1689 |         self.embs = nn.ModuleList([e for e in embs if e is not None])
1690 |         # Map from fid index to position in self.embs (or -1 if filtered)
1691 |         self._emb_index = []
1692 |         real_idx = 0
1693 |         for e in embs:
1694 |             if e is not None:
1695 |                 self._emb_index.append(real_idx)
1696 |                 real_idx += 1
1697 |             else:
1698 |                 self._emb_index.append(-1)
1699 |
1700 |         # Compute total embedding dim: sum of all fids across all groups
1701 |         total_num_fids = sum(len(g) for g in groups)
1702 |         total_emb_dim = total_num_fids * emb_dim
1703 |
1704 |         # Pad total_emb_dim to be divisible by num_ns_tokens
1705 |         self.chunk_dim = math.ceil(total_emb_dim / num_ns_tokens)
1706 |         self.padded_total_dim = self.chunk_dim * num_ns_tokens
1707 |         self._pad_size = self.padded_total_dim - total_emb_dim
1708 |
1709 |         # Per-chunk projection: chunk_dim -> d_model with LayerNorm
1710 |         self.token_projs = nn.ModuleList([
1711 |             nn.Sequential(
1712 |                 nn.Linear(self.chunk_dim, d_model),
1713 |                 nn.LayerNorm(d_model),
1714 |             )
1715 |             for _ in range(num_ns_tokens)
1716 |         ])
1717 |
1718 |         logging.info(
1719 |             f"RankMixerNSTokenizer: {total_num_fids} fids, "
1720 |             f"total_emb_dim={total_emb_dim}, chunk_dim={self.chunk_dim}, "
1721 |             f"num_ns_tokens={num_ns_tokens}, pad={self._pad_size}"
1722 |         )
1723 |
1724 |     def forward(self, int_feats: torch.Tensor) -> torch.Tensor:
1725 |         """Embeds all features, concatenates, splits, and projects.
1726 |
1727 |         Args:
1728 |             int_feats: (B, total_int_dim) concatenated integer features.
1729 |
1730 |         Returns:
1731 |             (B, num_ns_tokens, d_model) tensor.
1732 |         """
1733 |         # 1. Embed all fids in group order → flat cat
1734 |         all_embs = []
1735 |         for group in self.groups:
1736 |             for fid_idx in group:
1737 |                 vs, offset, length = self.feature_specs[fid_idx]
1738 |                 emb_real_idx = self._emb_index[fid_idx]
1739 |                 if emb_real_idx == -1:
1740 |                     fid_emb = int_feats.new_zeros(int_feats.shape[0], self.emb_dim)
1741 |                 else:
1742 |                     emb_layer = self.embs[emb_real_idx]
1743 |                     if length == 1:
1744 |                         fid_emb = emb_layer(int_feats[:, offset].long())
1745 |                     else:
1746 |                         vals = int_feats[:, offset:offset + length].long()
1747 |                         emb_all = emb_layer(vals)
1748 |                         mask = (vals != 0).float().unsqueeze(-1)
1749 |                         count = mask.sum(dim=1).clamp(min=1)
1750 |                         fid_emb = (emb_all * mask).sum(dim=1) / count
1751 |                 all_embs.append(fid_emb)
1752 |
1753 |         cat_emb = torch.cat(all_embs, dim=-1)  # (B, total_emb_dim)
1754 |
1755 |         # 2. Pad if needed
1756 |         if self._pad_size > 0:
1757 |             cat_emb = F.pad(cat_emb, (0, self._pad_size))  # (B, padded_total_dim)
1758 |
1759 |         # 3. Split into num_ns_tokens chunks and project each
1760 |         chunks = cat_emb.split(self.chunk_dim, dim=-1)  # list of (B, chunk_dim)
1761 |         tokens = []
1762 |         for chunk, proj in zip(chunks, self.token_projs):
1763 |             tokens.append(F.silu(proj(chunk)).unsqueeze(1))  # (B, 1, d_model)
1764 |
1765 |         return torch.cat(tokens, dim=1)  # (B, num_ns_tokens, d_model)
```

```
|
|
| class PCVRHyFormer(nn.Module):
|     """PCVRHyFormer model for post-click conversion rate prediction.
|
|     Combines MultiSeqHyFormerBlock and MultiSeqQueryGenerator to process
|     multiple input sequences with non-sequence features.
|     """
|
|     def __init__(
|         self,
|         # Data schema
|         user_int_feature_specs: List[Tuple[int, int, int]],
|         item_int_feature_specs: List[Tuple[int, int, int]],
|         user_dense_dim: int,
|         item_dense_dim: int,
|         seq_vocab_sizes: "dict[str, List[int]]",  # {domain: [vocab_size_per_fid, ...]}
|         # NS grouping config (grouped by fid index)
|         user_ns_groups: List[List[int]],
|         item_ns_groups: List[List[int]],
|         # Model hyperparameters
|         d_model: int = 64,
|         emb_dim: int = 64,
|         num_queries: int = 1,
|         num_hyformer_blocks: int = 2,
|         num_heads: int = 4,
|         seq_encoder_type: str = 'transformer',
|         hidden_mult: int = 4,
|         dropout_rate: float = 0.01,
|         seq_top_k: int = 50,
|         seq_causal: bool = False,
|         action_num: int = 1,
|         num_time_buckets: int = 65,
|         rank_mixer_mode: str = 'full',
|         use_rope: bool = False,
|         rope_base: float = 10000.0,
|         emb_skip_threshold: int = 0,
|         seq_id_threshold: int = 10000,
|         # NS tokenizer variant
|         ns_tokenizer_type: str = 'rankmixer',
|         user_ns_tokens: int = 0,
|         item_ns_tokens: int = 0,
|         # OneTrans unified attention (replaces HyFormer query-decoding blocks)
|         use_onetrans: bool = False,
|         onetrans_top_k: int = 50,
|         # T13: hash embedding for sequence features with vocab > emb_skip_threshold
|         # (0 = disabled, use zero vector for skipped features as before)
|         seq_hash_vocab: int = 0,
|         # Item-conditioned query generation (DIN-style target-item awareness)
|         item_conditioned_query: bool = False,
|         # ADR-004 · Target-item hard interaction injected into seq tokens
|         # (seq-token level target-item conditioning, orthogonal to ICQ which
|         # is query-side). Default 'off' keeps behaviour identical to baseline.
|         target_item_seq_injection: str = 'off',
|         target_inject_alpha_init: float = 0.0,
|         # Dense bypass: project dense features directly to the classifier input,
|         # bypassing NS-token dilution in HyFormer blocks. When enabled the dense
|         # token is removed from ns_tokens and the classifier input dimension is
|         # extended by d_model (d_model + d_model = 2*d_model).
|         enable_dense_bypass: bool = False,
|     ) -> None:
|         super().__init__()
|
|         self.d_model = d_model
|         self.emb_dim = emb_dim
|         self.action_num = action_num
|         self.num_queries = num_queries
|         self.seq_domains = sorted(seq_vocab_sizes.keys())  # deterministic order
|         self.enable_dense_bypass = enable_dense_bypass
|         self.num_sequences = len(self.seq_domains)
|         self.num_time_buckets = num_time_buckets
|         self.rank_mixer_mode = rank_mixer_mode
|         self.use_rope = use_rope
|         self.emb_skip_threshold = emb_skip_threshold
|         self.seq_id_threshold = seq_id_threshold
|         self.ns_tokenizer_type = ns_tokenizer_type
|         self.use_onetrans = use_onetrans
|         self.onetrans_top_k = onetrans_top_k
|         self.seq_hash_vocab = seq_hash_vocab
|         self.target_item_seq_injection = target_item_seq_injection
|         self.target_inject_alpha_init = target_inject_alpha_init
|
|         # ================== NS Tokens Construction ==================
|
|         if ns_tokenizer_type == 'group':
|             # Original: one NS token per group
|             self.user_ns_tokenizer = GroupNSTokenizer(
|                 feature_specs=user_int_feature_specs,
|                 groups=user_ns_groups,
|                 emb_dim=emb_dim,
```

```
                d_model=d_model,
                emb_skip_threshold=emb_skip_threshold,
            )
            num_user_ns = len(user_ns_groups)

            self.item_ns_tokenizer = GroupNSTokenizer(
                feature_specs=item_int_feature_specs,
                groups=item_ns_groups,
                emb_dim=emb_dim,
                d_model=d_model,
                emb_skip_threshold=emb_skip_threshold,
            )
            num_item_ns = len(item_ns_groups)
        elif ns_tokenizer_type == 'rankmixer':
            # RankMixer paper style: all embeddings cat → split → project
            # 0 means auto: fall back to group count
            if user_ns_tokens <= 0:
                user_ns_tokens = len(user_ns_groups)
            if item_ns_tokens <= 0:
                item_ns_tokens = len(item_ns_groups)
            self.user_ns_tokenizer = RankMixerNSTokenizer(
                feature_specs=user_int_feature_specs,
                groups=user_ns_groups,
                emb_dim=emb_dim,
                d_model=d_model,
                num_ns_tokens=user_ns_tokens,
                emb_skip_threshold=emb_skip_threshold,
            )
            num_user_ns = user_ns_tokens

            self.item_ns_tokenizer = RankMixerNSTokenizer(
                feature_specs=item_int_feature_specs,
                groups=item_ns_groups,
                emb_dim=emb_dim,
                d_model=d_model,
                num_ns_tokens=item_ns_tokens,
                emb_skip_threshold=emb_skip_threshold,
            )
            num_item_ns = item_ns_tokens
        else:
            raise ValueError(f"Unknown ns_tokenizer_type: {ns_tokenizer_type}")

        # Save for downstream modules that need to slice item NS tokens out
        # of the concatenated ns_tokens tensor (e.g. ICQ, target injector).
        self.num_item_ns = num_item_ns
        self.num_user_ns = num_user_ns

        # User dense feature projection (if available)
        self.has_user_dense = user_dense_dim > 0
        if self.has_user_dense:
            self.user_dense_proj = nn.Sequential(
                nn.Linear(user_dense_dim, d_model),
                nn.LayerNorm(d_model),
            )

        # Item dense feature projection (if available)
        self.has_item_dense = item_dense_dim > 0
        if self.has_item_dense:
            self.item_dense_proj = nn.Sequential(
                nn.Linear(item_dense_dim, d_model),
                nn.LayerNorm(d_model),
            )

        # Total NS token count: bypass mode keeps the dense NS token so that
        # the rank_mixer T-divisibility constraint (d_model % T == 0) is
        # preserved. The bypass projection adds a SECOND, undiluted path for
        # the dense features directly into the classifier — they flow through
        # both the NS-token path and the bypass path simultaneously.
        self.num_ns = (num_user_ns + (1 if self.has_user_dense else 0)
                       + num_item_ns + (1 if self.has_item_dense else 0))

        # Dense bypass projections: an extra projection added ALONGSIDE the
        # NS-token path when enable_dense_bypass=True. The projected vector is
        # concatenated directly to the output of output_proj before the
        # classifier, giving the dense features a full-strength signal path
        # independent of the HyFormer block dilution.
        self.has_dense_bypass = enable_dense_bypass and self.has_user_dense
        if self.has_dense_bypass:
            self.user_dense_bypass_proj = nn.Sequential(
                nn.Linear(user_dense_dim, d_model),
                nn.LayerNorm(d_model),
                nn.SiLU(),
            )

        # ================== Check d_model % T == 0 constraint (full mode only) ==================
        T = num_queries * self.num_sequences + self.num_ns
        if rank_mixer_mode == 'full' and d_model % T != 0 and not use_onetrans:
            valid_T_values = [t for t in range(1, d_model + 1) if d_model % t == 0]
            raise ValueError(
                f"d_model={d_model} must be divisible by T=num_queries*num_sequences+num_ns="
```

```
                f"{num_queries}*{self.num_sequences}+{self.num_ns}={T}. "
                f"Valid T values for d_model={d_model}: {valid_T_values}"
            )

        # ================= Seq Tokens Embedding ==================
        # seq_id_threshold decides which features inside the seq tokenizer are
        # treated as id features (they receive extra dropout). It is fully
        # independent of emb_skip_threshold (which skips Embedding creation).
        self.seq_id_emb_dropout = nn.Dropout(dropout_rate * 2)

        def _make_seq_embs(vocab_sizes):
            """Create embedding list for sequence domain features.

            Features with vocab_size <= 0 → None (zero vector).
            Features with vocab_size > emb_skip_threshold → None unless
            seq_hash_vocab > 0, in which case a hash embedding of size
            seq_hash_vocab is created and values are hashed modulo
            seq_hash_vocab at forward time (T13 fix for domain_c_seq_47).
            """
            embs_raw = []
            hash_flags = []
            for vs in vocab_sizes:
                if int(vs) <= 0:
                    embs_raw.append(None)
                    hash_flags.append(False)
                elif emb_skip_threshold > 0 and int(vs) > emb_skip_threshold:
                    if seq_hash_vocab > 0:
                        # Hash embedding: recover high-vocab item IDs via modulo
                        embs_raw.append(nn.Embedding(
                            int(seq_hash_vocab) + 1, emb_dim, padding_idx=0))
                        hash_flags.append(True)
                    else:
                        embs_raw.append(None)
                        hash_flags.append(False)
                else:
                    embs_raw.append(nn.Embedding(int(vs) + 1, emb_dim, padding_idx=0))
                    hash_flags.append(False)
            module_list = nn.ModuleList([e for e in embs_raw if e is not None])
            index_map = []
            real_idx = 0
            for e in embs_raw:
                if e is not None:
                    index_map.append(real_idx)
                    real_idx += 1
                else:
                    index_map.append(-1)
            is_id = [int(vs) > seq_id_threshold for vs in vocab_sizes]
            return module_list, index_map, is_id, hash_flags

        # ================= Dynamic Sequence Embeddings ==================
        self._seq_embs = nn.ModuleDict()
        self._seq_emb_index = {}    # domain -> index_map
        self._seq_is_id = {}        # domain -> is_id list
        self._seq_hash_flags = {}   # domain -> hash_flags (True = modulo hash)
        self._seq_vocab_sizes = {}  # domain -> vocab_sizes list
        self._seq_proj = nn.ModuleDict()

        for domain in self.seq_domains:
            vs = seq_vocab_sizes[domain]
            embs, idx_map, is_id, hash_flags = _make_seq_embs(vs)
            self._seq_embs[domain] = embs
            self._seq_emb_index[domain] = idx_map
            self._seq_is_id[domain] = is_id
            self._seq_hash_flags[domain] = hash_flags
            self._seq_vocab_sizes[domain] = vs
            self._seq_proj[domain] = nn.Sequential(
                nn.Linear(len(vs) * emb_dim, d_model),
                nn.LayerNorm(d_model),
            )

        # ================= Time Interval Bucket Embedding (optional) ==================
        if num_time_buckets > 0:
            self.time_embedding = nn.Embedding(num_time_buckets, d_model, padding_idx=0)

        # ================= HyFormer OR OneTrans Components ==================
        # Target-item seq injector (ADR-004) — applied to seq_tokens_list
        # BEFORE the HyFormer / OneTrans path, so it benefits both. When
        # target_item_seq_injection='off' this is a zero-param identity.
        self.target_item_seq_injector = TargetItemSeqInjector(
            d_model=d_model,
            num_sequences=self.num_sequences,
            mode=target_item_seq_injection,
            alpha_init=target_inject_alpha_init,
        )

        if not use_onetrans:
            # ---- Classic HyFormer path ----
            self.query_generator = MultiSeqQueryGenerator(
                d_model=d_model,
                num_ns=self.num_ns,
```

```
                num_queries=num_queries,
                num_sequences=self.num_sequences,
                hidden_mult=hidden_mult,
                item_conditioned=item_conditioned_query,
                num_item_ns=num_item_ns,
            )
            self.blocks = nn.ModuleList([
                MultiSeqHyFormerBlock(
                    d_model=d_model,
                    num_heads=num_heads,
                    num_queries=num_queries,
                    num_ns=self.num_ns,
                    num_sequences=self.num_sequences,
                    seq_encoder_type=seq_encoder_type,
                    hidden_mult=hidden_mult,
                    dropout=dropout_rate,
                    top_k=seq_top_k,
                    causal=seq_causal,
                    rank_mixer_mode=rank_mixer_mode,
                )
                for _ in range(num_hyformer_blocks)
            ])
            # Output: concat Q tokens from all domains → project to d_model
            self.output_proj = nn.Sequential(
                nn.Linear(num_queries * self.num_sequences * d_model, d_model),
                nn.LayerNorm(d_model),
            )
        else:
            # ---- OneTrans path ----
            # Per-domain lightweight SwiGLU encoder (processes sequences before
            # top-K compression; shared across OneTrans blocks for efficiency)
            self.ot_seq_encoders = nn.ModuleDict({
                domain: SwiGLUEncoder(d_model, hidden_mult, dropout_rate)
                for domain in self.seq_domains
            })
            # Stack of OneTrans blocks
            self.onetrans_blocks = nn.ModuleList([
                OneTransBlock(
                    d_model=d_model,
                    num_heads=num_heads,
                    num_ns=self.num_ns,
                    hidden_mult=hidden_mult,
                    dropout=dropout_rate,
                )
                for _ in range(num_hyformer_blocks)
            ])
            # NS-token sequence conditioner: gives NS tokens initial sequence
            # context before the first unified attention (prevents cold-start)
            self.ot_ns_conditioner = OneTransNSConditioner(
                d_model=d_model,
                num_ns=self.num_ns,
                num_sequences=self.num_sequences,
                hidden_mult=2,
            )
            # Output: NS tokens flattened → project to d_model
            self.output_proj = nn.Sequential(
                nn.Linear(self.num_ns * d_model, d_model),
                nn.LayerNorm(d_model),
            )
            # No query_generator or HyFormer blocks needed
            self.query_generator = None
            self.blocks = nn.ModuleList()

        # ================== RoPE (HyFormer path only) ==================
        if use_rope and not use_onetrans:
            head_dim = d_model // num_heads
            self.rotary_emb = RotaryEmbedding(dim=head_dim, base=rope_base)
        else:
            self.rotary_emb = None

        # Dropout
        self.emb_dropout = nn.Dropout(dropout_rate)

        # Classifier input dimension: d_model normally; d_model*2 when dense
        # bypass is active because the bypass vector is concatenated to output.
        clsfier_in_dim = d_model * 2 if self.has_dense_bypass else d_model

        # Classifier
        self.clsfier = nn.Sequential(
            nn.Linear(clsfier_in_dim, d_model),
            nn.LayerNorm(d_model),
            nn.SiLU(),
            nn.Dropout(dropout_rate),
            nn.Linear(d_model, action_num)
        )

        # Initialize parameters
        self._init_params()

        # Log emb_skip_threshold filtering stats
```

```
        if emb_skip_threshold > 0:
            def _count_filtered(vocab_sizes, emb_index):
                filtered = sum(1 for idx in emb_index if idx == -1)
                return filtered, len(vocab_sizes)
            for domain in self.seq_domains:
                f, t = _count_filtered(self._seq_vocab_sizes[domain], self._seq_emb_index[domain])
                if f > 0:
                    logging.info(f"emb_skip_threshold={emb_skip_threshold}: {domain} skipped {f}/{t} features")
            for name, tokenizer in [
                ("user_ns", self.user_ns_tokenizer),
                ("item_ns", self.item_ns_tokenizer),
            ]:
                f = sum(1 for idx in tokenizer._emb_index if idx == -1)
                t = len(tokenizer._emb_index)
                if f > 0:
                    logging.info(f"emb_skip_threshold={emb_skip_threshold}: {name} skipped {f}/{t} features")

    def _init_params(self) -> None:
        """Applies Xavier initialization to all embedding weights."""
        for domain in self.seq_domains:
            for emb in self._seq_embs[domain]:
                nn.init.xavier_normal_(emb.weight.data)
                emb.weight.data[0, :] = 0

        for tokenizer in [self.user_ns_tokenizer, self.item_ns_tokenizer]:
            for emb in tokenizer.embs:
                nn.init.xavier_normal_(emb.weight.data)
                emb.weight.data[0, :] = 0

        if self.num_time_buckets > 0:
            nn.init.xavier_normal_(self.time_embedding.weight.data)
            self.time_embedding.weight.data[0, :] = 0

    def reinit_high_cardinality_params(
        self, cardinality_threshold: int = 10000
    ) -> "set[int]":
        """Reinitializes only high-cardinality embeddings.

        Preserves low-cardinality and time feature embeddings.

        Args:
            cardinality_threshold: Only embeddings with vocab_size exceeding
                this value are reinitialized.

        Returns:
            A set of data_ptr() values for reinitialized parameters.
        """
        reinit_count = 0
        skip_count = 0
        reinit_ptrs = set()

        for emb_list, vocab_sizes, emb_index in [
            (self._seq_embs[d], self._seq_vocab_sizes[d], self._seq_emb_index[d])
            for d in self.seq_domains
        ]:
            for i, vs in enumerate(vocab_sizes):
                real_idx = emb_index[i]
                if real_idx == -1:
                    # Skipped by emb_skip_threshold, no embedding to reinit
                    continue
                emb = emb_list[real_idx]
                if int(vs) > cardinality_threshold:
                    nn.init.xavier_normal_(emb.weight.data)
                    emb.weight.data[0, :] = 0
                    reinit_ptrs.add(emb.weight.data_ptr())
                    reinit_count += 1
                else:
                    skip_count += 1

        for tokenizer, specs in [
            (self.user_ns_tokenizer, self.user_ns_tokenizer.feature_specs),
            (self.item_ns_tokenizer, self.item_ns_tokenizer.feature_specs),
        ]:
            for i, (vs, offset, length) in enumerate(specs):
                real_idx = tokenizer._emb_index[i]
                if real_idx == -1:
                    continue
                emb = tokenizer.embs[real_idx]
                if int(vs) > cardinality_threshold:
                    nn.init.xavier_normal_(emb.weight.data)
                    emb.weight.data[0, :] = 0
                    reinit_ptrs.add(emb.weight.data_ptr())
                    reinit_count += 1
                else:
                    skip_count += 1

        # time_embedding is always preserved
        if self.num_time_buckets > 0:
            skip_count += 1

```

```
        logging.info(f"Re-initialized {reinit_count} high-cardinality Embeddings "
                     f"(vocab>{cardinality_threshold}), kept {skip_count}")
        return reinit_ptrs

    def get_sparse_params(self) -> List[nn.Parameter]:
        """Returns all embedding table parameters (optimized with Adagrad)."""
        sparse_params = set()
        for module in self.modules():
            if isinstance(module, nn.Embedding):
                sparse_params.add(module.weight.data_ptr())
        return [p for p in self.parameters() if p.data_ptr() in sparse_params]

    def get_dense_params(self) -> List[nn.Parameter]:
        """Returns all non-embedding parameters (optimized with AdamW)."""
        sparse_ptrs = {p.data_ptr() for p in self.get_sparse_params()}
        return [p for p in self.parameters() if p.data_ptr() not in sparse_ptrs]

    def _embed_seq_domain(
        self,
        seq: torch.Tensor,
        sideinfo_embs: nn.ModuleList,
        proj: nn.Module,
        is_id: List[bool],
        emb_index: List[int],
        time_bucket_ids: torch.Tensor,
        hash_flags: Optional[List[bool]] = None,
    ) -> torch.Tensor:
        """Embeds a sequence domain by concatenating sideinfo embeddings and projecting to d_model."""
        B, S, L = seq.shape
        emb_list = []
        for i in range(S):
            real_idx = emb_index[i] if i < len(emb_index) else -1
            if real_idx == -1:
                emb_list.append(seq.new_zeros(B, L, self.emb_dim, dtype=torch.float))
            else:
                emb = sideinfo_embs[real_idx]
                vals = seq[:, i, :]  # (B, L)
                # Hash embedding: map high-vocab item IDs via modulo
                if hash_flags is not None and i < len(hash_flags) and hash_flags[i]:
                    # Non-zero values get hashed; zero (padding) stays 0
                    nonzero = vals != 0
                    vals = vals.clone()
                    vals[nonzero] = (vals[nonzero] % self.seq_hash_vocab).clamp(min=1)
                e = emb(vals)  # (B, L, emb_dim)
                if is_id[i] and self.training:
                    e = self.seq_id_emb_dropout(e)
                emb_list.append(e)
        cat_emb = torch.cat(emb_list, dim=-1)  # (B, L, S*emb_dim)
        token_emb = F.gelu(proj(cat_emb))  # (B, L, D)

        if self.num_time_buckets > 0:
            token_emb = token_emb + self.time_embedding(time_bucket_ids)

        return token_emb

    def _make_padding_mask(
        self, seq_len: torch.Tensor, max_len: int
    ) -> torch.Tensor:
        """Generates a padding mask from sequence lengths."""
        device = seq_len.device
        idx = torch.arange(max_len, device=device).unsqueeze(0)  # (1, max_len)
        return idx >= seq_len.unsqueeze(1)  # (B, max_len)

    def _gather_onetrans_topk(
        self,
        seq_tokens: torch.Tensor,
        padding_mask: torch.Tensor,
    ) -> Tuple[torch.Tensor, torch.Tensor]:
        """Take the last top_k valid tokens per sample (like LongerEncoder).

        Args:
            seq_tokens: (B, L, D)
            padding_mask: (B, L) True = padding

        Returns:
            topk_tokens: (B, K, D)
            topk_mask: (B, K) True = padding
        """
        B, L, D = seq_tokens.shape
        K = min(self.onetrans_top_k, L)
        valid_len = (~padding_mask).sum(dim=1)  # (B,)
        actual_k = valid_len.clamp(max=K)
        start_pos = (valid_len - actual_k).clamp(min=0)

        device = seq_tokens.device
        offsets = torch.arange(K, device=device).unsqueeze(0).expand(B, -1)
        indices = (start_pos.unsqueeze(1) + offsets).clamp(0, L - 1)
        indices_exp = indices.unsqueeze(-1).expand(-1, -1, D)
        topk_tokens = torch.gather(seq_tokens, 1, indices_exp)  # (B, K, D)

```

```
        pos_idx = torch.arange(K, device=device).unsqueeze(0)
        topk_mask = pos_idx >= actual_k.unsqueeze(1)  # (B, K) True = pad
        topk_tokens = topk_tokens * (~topk_mask).unsqueeze(-1).float()
        return topk_tokens, topk_mask

    def _run_onetrans_blocks(
        self,
        seq_tokens_list: list,
        seq_masks_list: list,
        ns_tokens: torch.Tensor,
        apply_dropout: bool = True,
    ) -> torch.Tensor:
        """OneTrans forward path.

        1. Per-domain lightweight SwiGLU encoding.
        2. Top-K compression per domain.
        3. Concatenate all domains into one S-token sequence.
        4. Stack of OneTransBlock layers.
        5. Flatten NS tokens → output_proj → d_model.
        """
        if apply_dropout:
            ns_tokens = self.emb_dropout(ns_tokens)
            seq_tokens_list = [self.emb_dropout(s) for s in seq_tokens_list]

        # Per-domain SwiGLU encoding (within-domain local context)
        compressed_s = []
        compressed_masks = []
        for i, domain in enumerate(self.seq_domains):
            enc_out, mask_out = self.ot_seq_encoders[domain](
                seq_tokens_list[i], seq_masks_list[i])
            topk_tok, topk_mask = self._gather_onetrans_topk(enc_out, mask_out)
            compressed_s.append(topk_tok)
            compressed_masks.append(topk_mask)

        # Concatenate all domains into unified S sequence
        s_tokens = torch.cat(compressed_s, dim=1)          # (B, K*S, D)
        s_mask = torch.cat(compressed_masks, dim=1)         # (B, K*S) True=pad

        # Stack OneTrans blocks
        # Condition NS tokens on sequence context before the first block
        ns_tokens = self.ot_ns_conditioner(ns_tokens, compressed_s, compressed_masks)

        for block in self.onetrans_blocks:
            s_tokens, ns_tokens = block(s_tokens, ns_tokens, s_mask)

        # Output: NS tokens (final representations) → project to d_model
        B = ns_tokens.shape[0]
        ns_flat = ns_tokens.view(B, -1)          # (B, M*D)
        output = self.output_proj(ns_flat)        # (B, D)
        return output

    def _run_multi_seq_blocks(
        self,
        q_tokens_list: list,
        ns_tokens: torch.Tensor,
        seq_tokens_list: list,
        seq_masks_list: list,
        apply_dropout: bool = True
    ) -> torch.Tensor:
        """Runs the HyFormer multi-sequence block stack."""
        if apply_dropout:
            q_tokens_list = [self.emb_dropout(q) for q in q_tokens_list]
            ns_tokens = self.emb_dropout(ns_tokens)
            seq_tokens_list = [self.emb_dropout(s) for s in seq_tokens_list]

        curr_qs = q_tokens_list
        curr_ns = ns_tokens
        curr_seqs = seq_tokens_list
        curr_masks = seq_masks_list

        for block in self.blocks:
            rope_cos_list = None
            rope_sin_list = None
            if self.rotary_emb is not None:
                rope_cos_list = []
                rope_sin_list = []
                device = curr_seqs[0].device
                for seq_i in curr_seqs:
                    seq_len = seq_i.shape[1]
                    cos, sin = self.rotary_emb(seq_len, device)
                    rope_cos_list.append(cos)
                    rope_sin_list.append(sin)

            curr_qs, curr_ns, curr_seqs, curr_masks = block(
                q_tokens_list=curr_qs,
                ns_tokens=curr_ns,
                seq_tokens_list=curr_seqs,
                seq_padding_masks=curr_masks,
                rope_cos_list=rope_cos_list,
                rope_sin_list=rope_sin_list,
```

```
2396 |             )
```